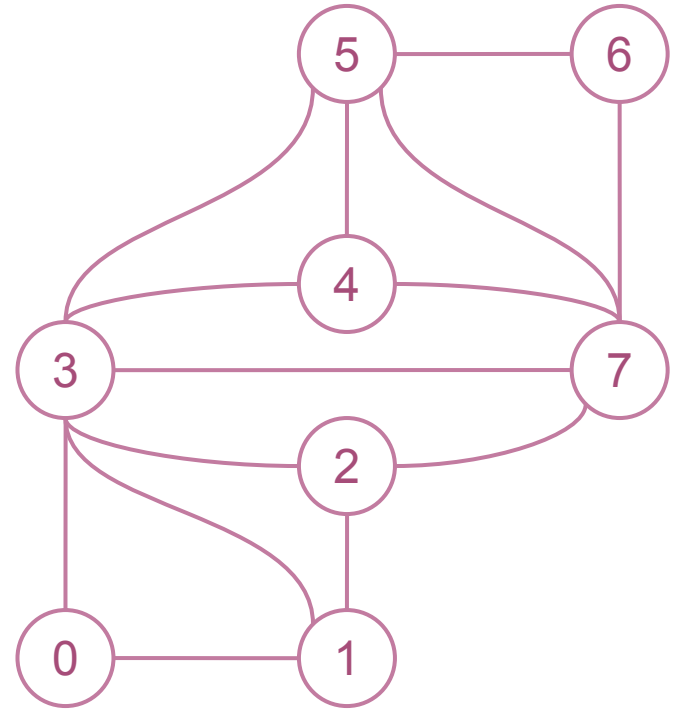
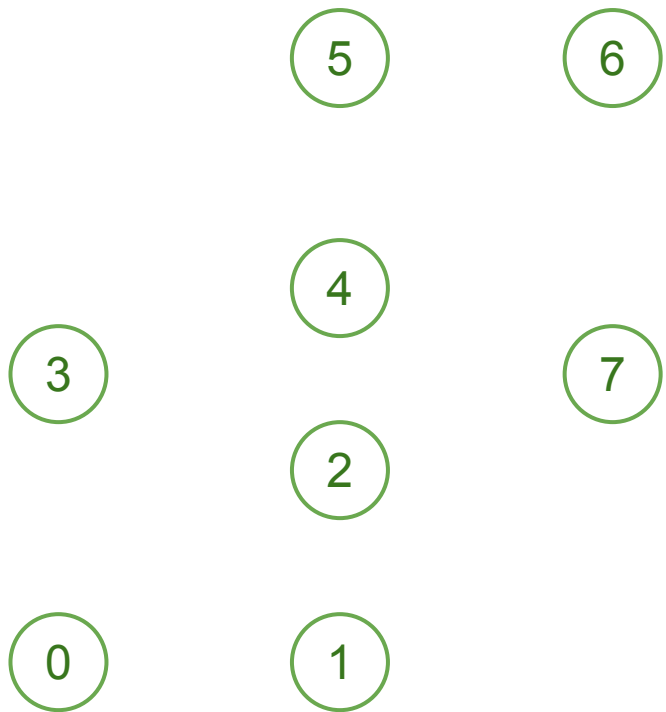


Minimum Spanning Tree

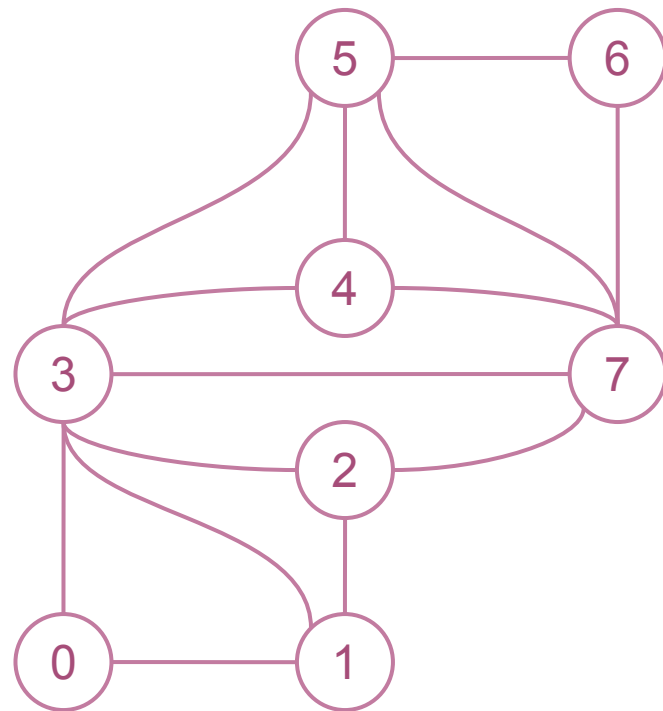
Input graph



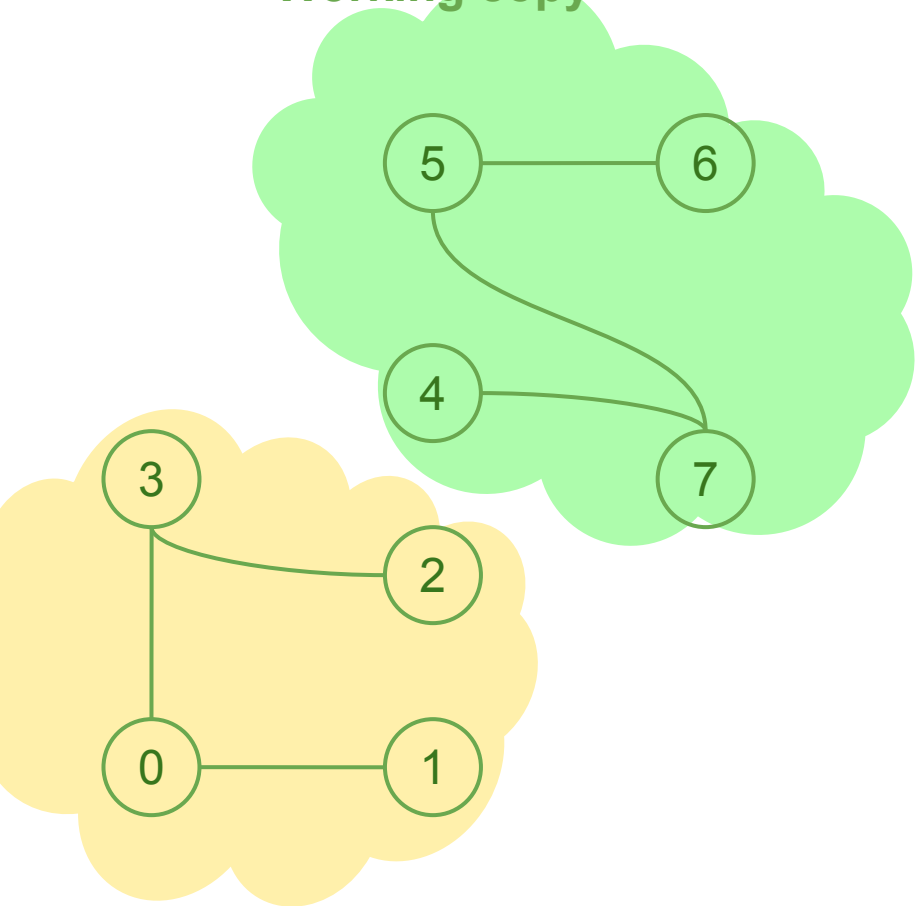
Working copy



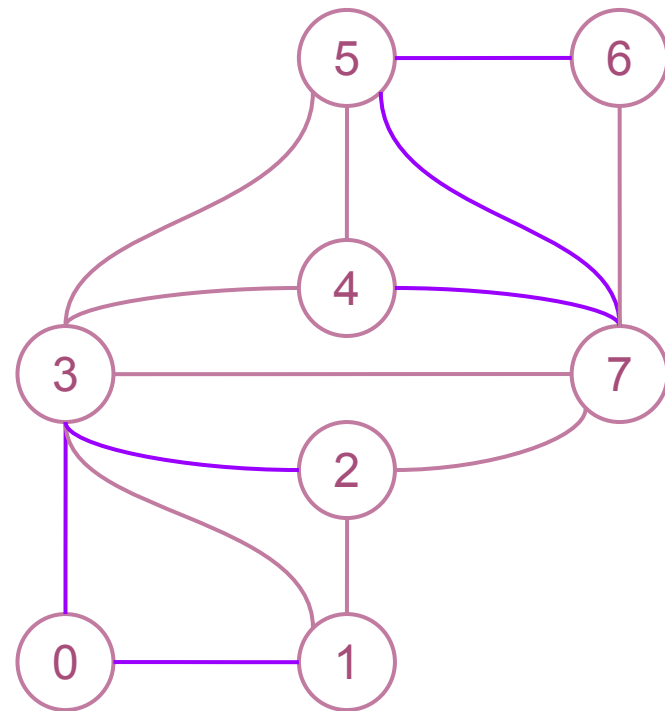
Input graph



Working copy

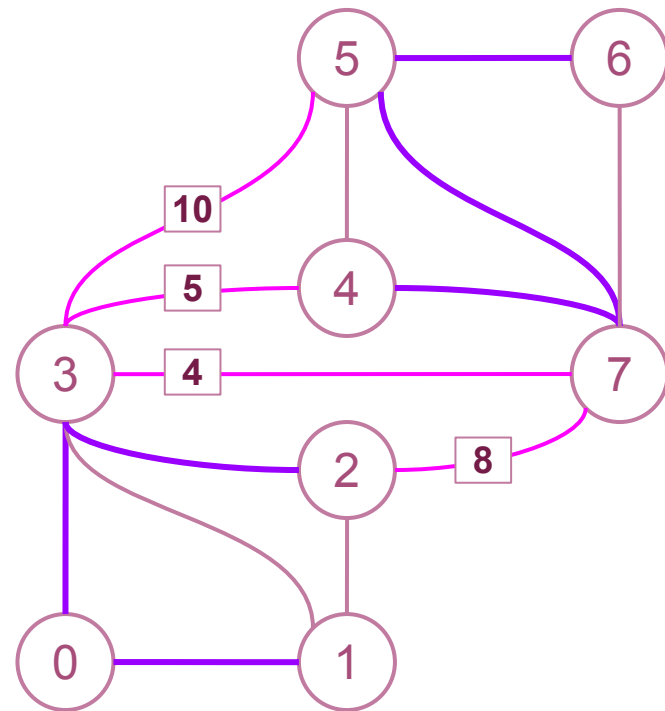
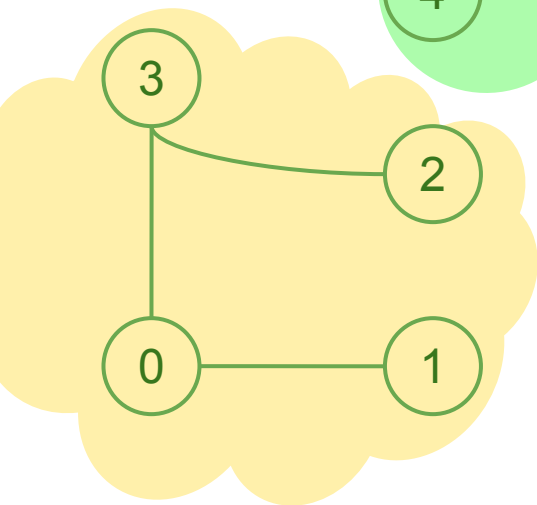


Input graph

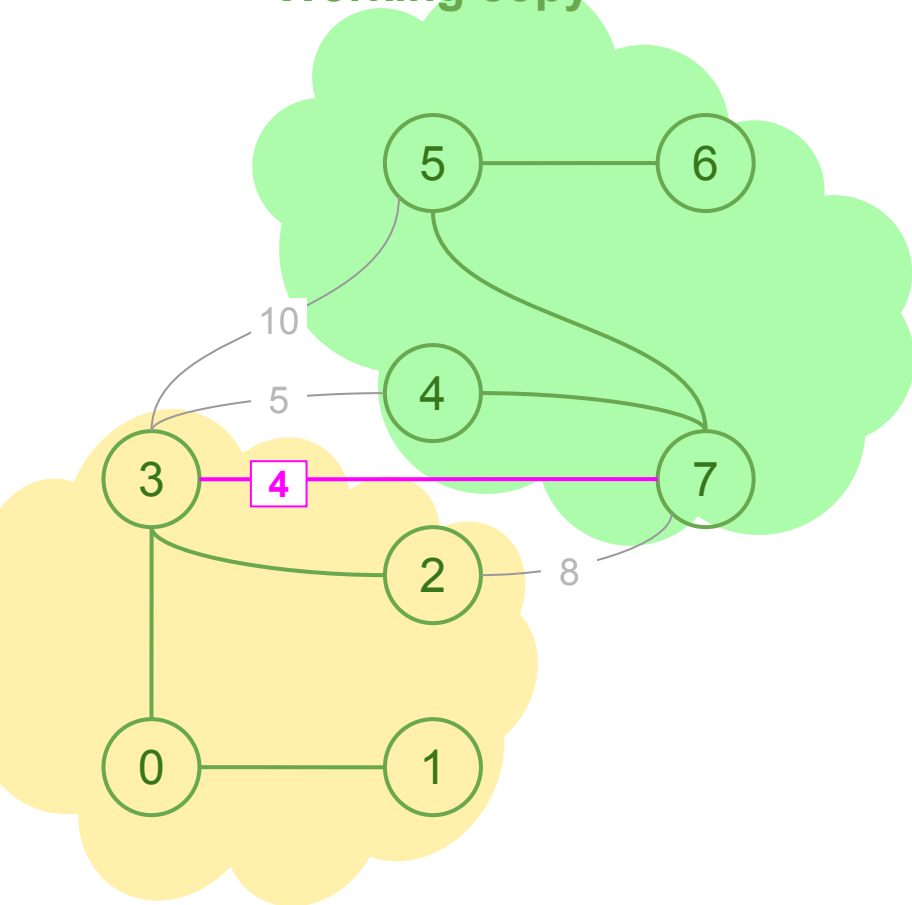


```

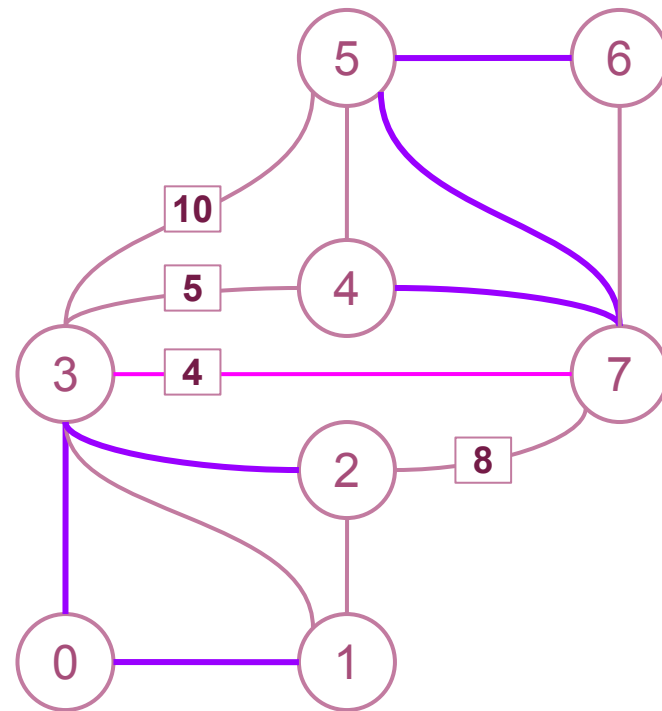
graph LR
    5((5)) --- 6((6))
    5((5)) --- 7((7))
    4((4)) --- 7((7))
  
```



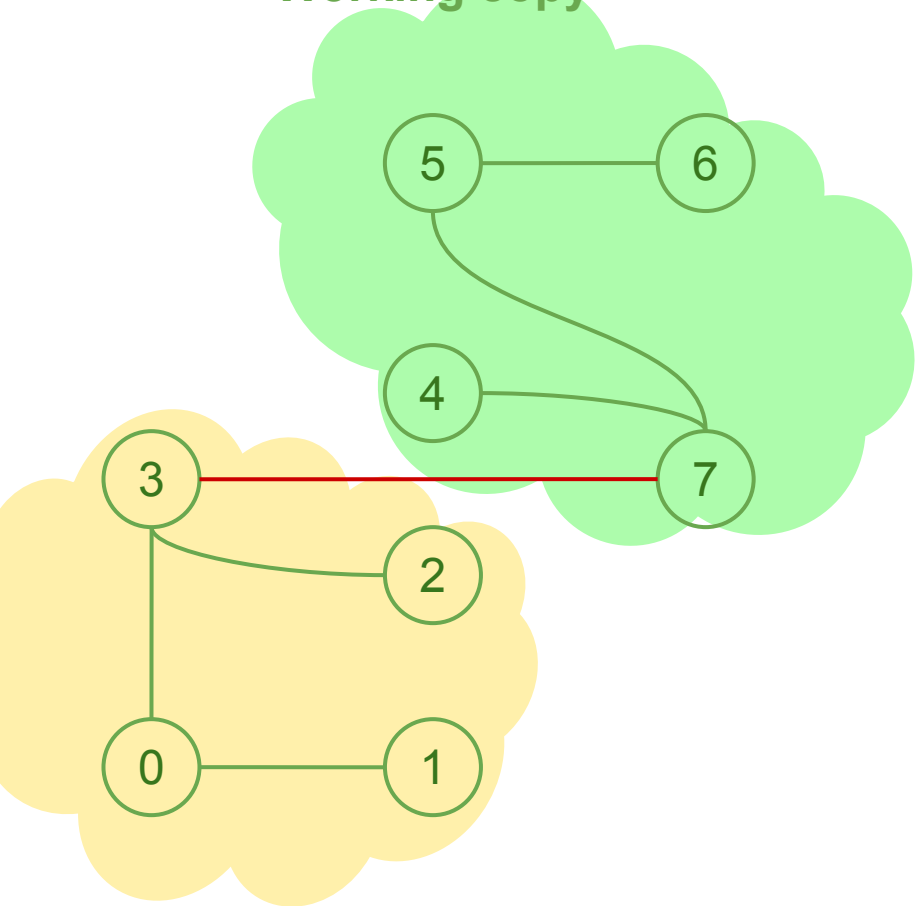
Working copy



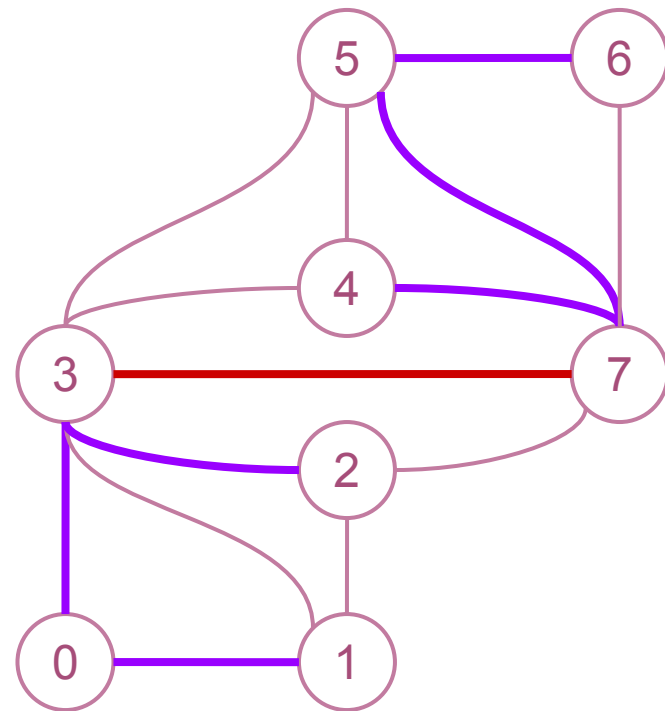
Input graph



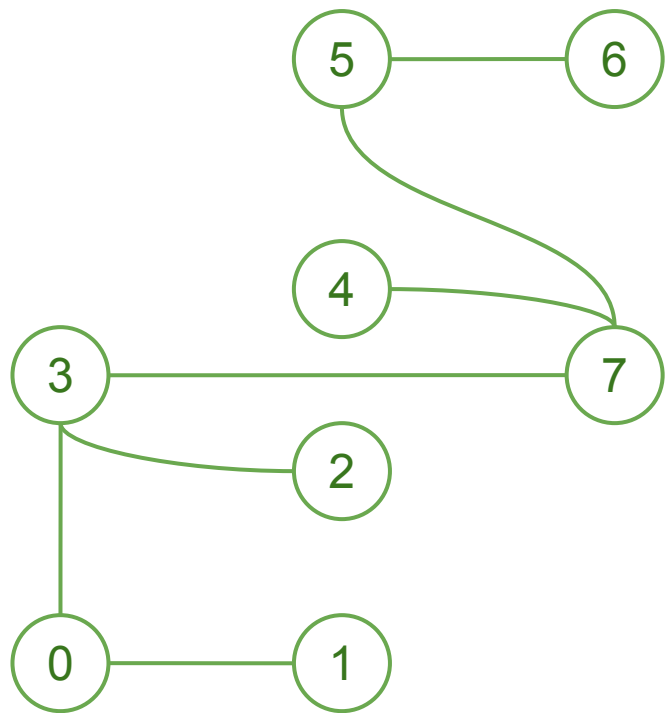
Working copy



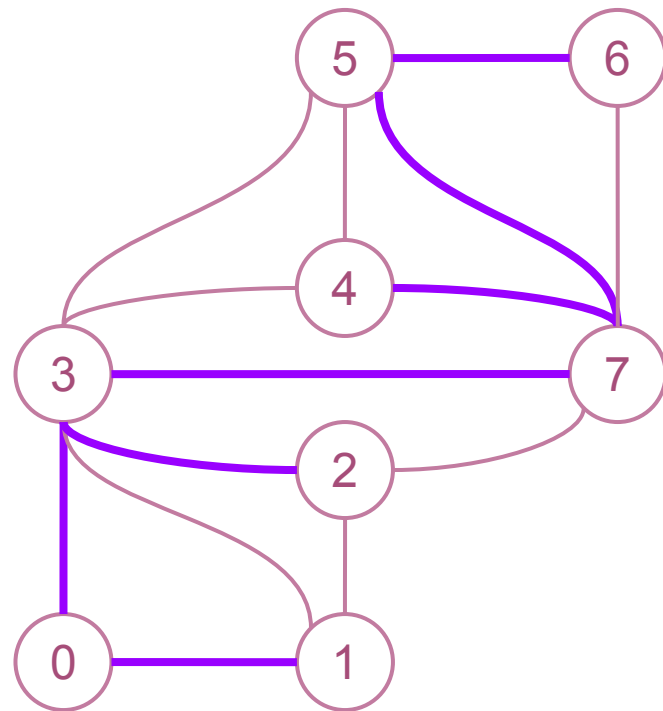
Input graph



Working copy

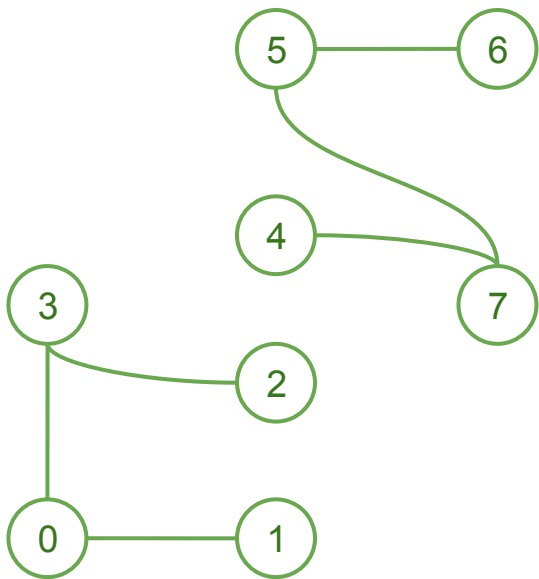


Input graph

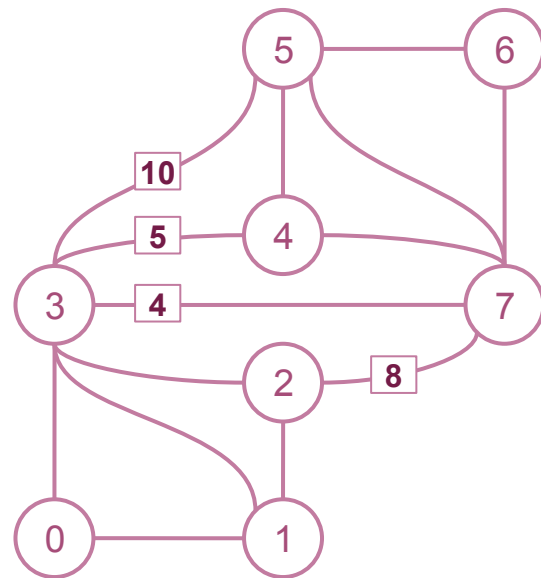


How to make it work?

Working copy

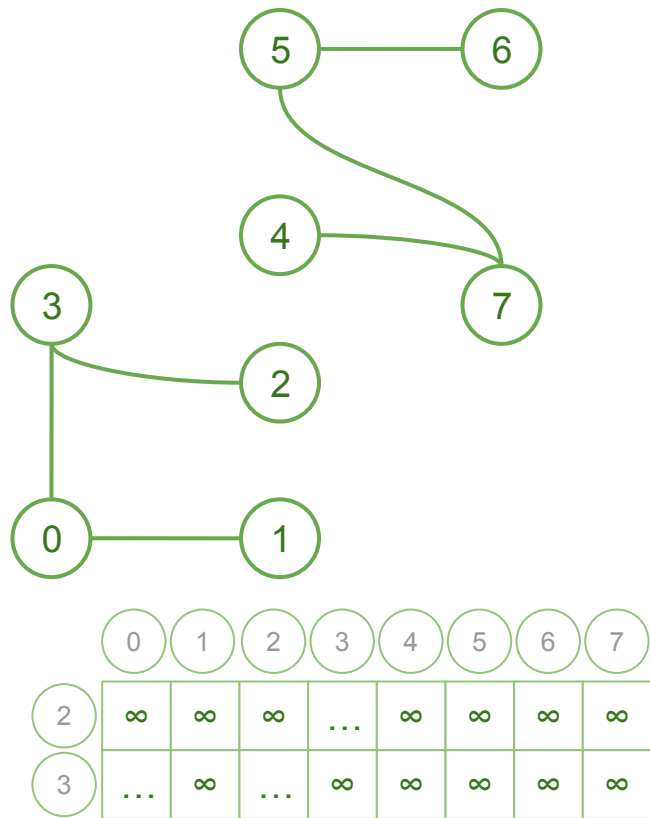


Input graph

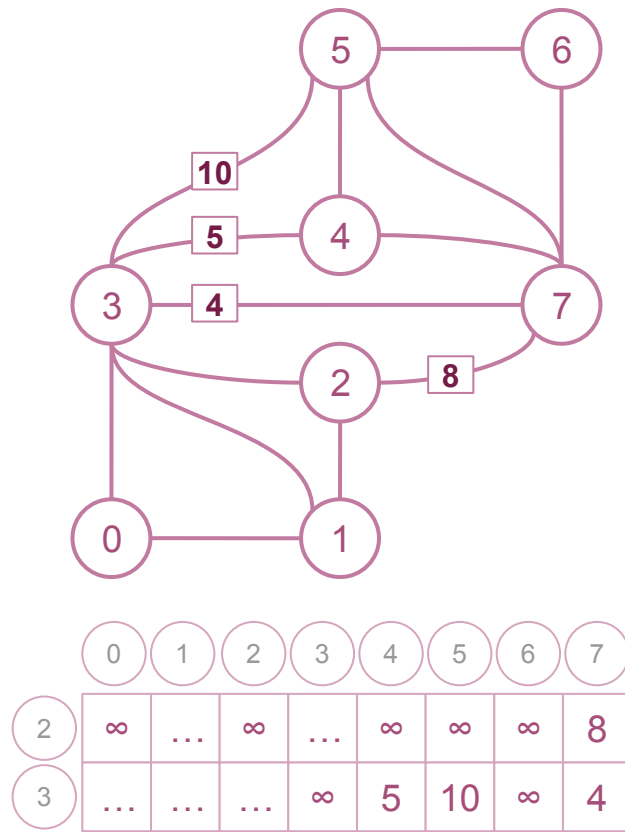


	0	1	2	3	4	5	6	7
2	∞	...	∞	...	∞	∞	∞	8
3	...	...	...	∞	5	10	∞	4

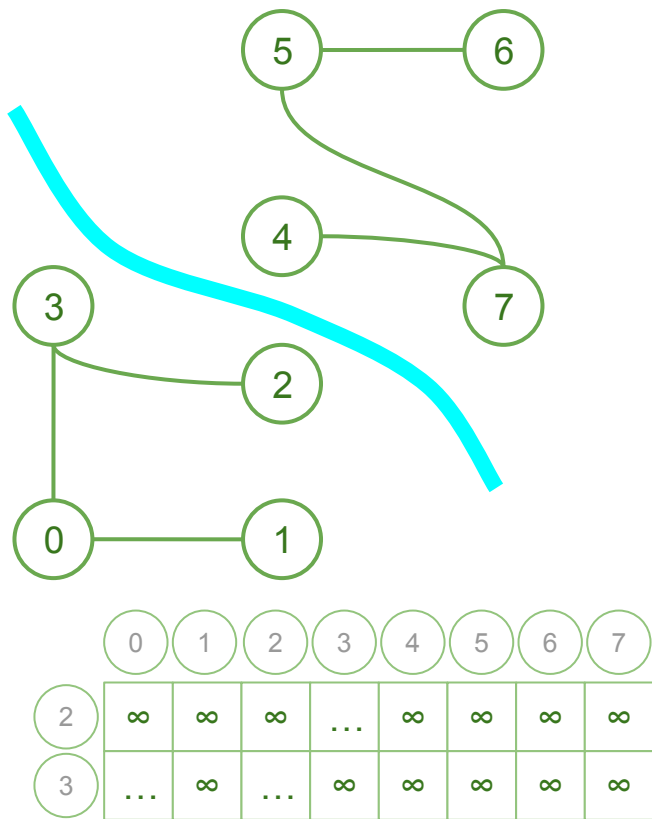
Working copy



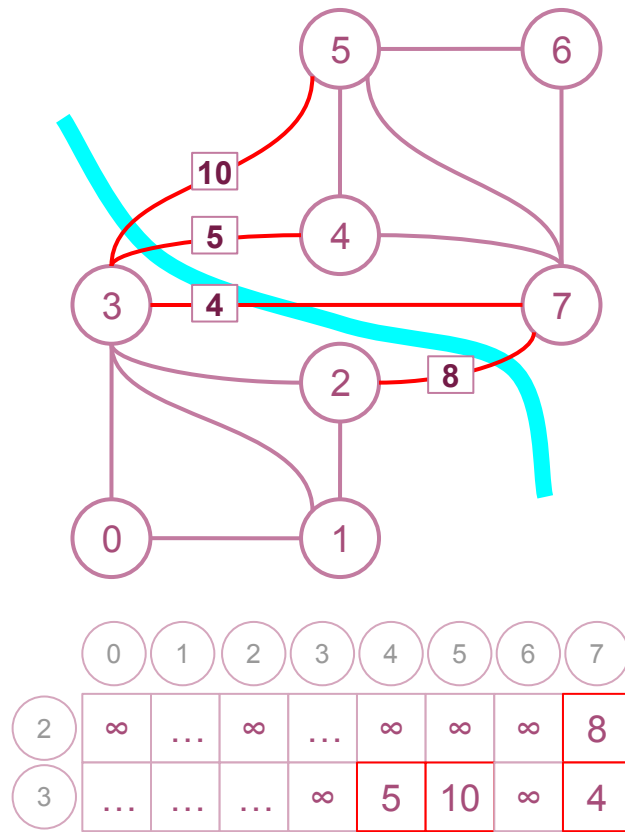
Input graph



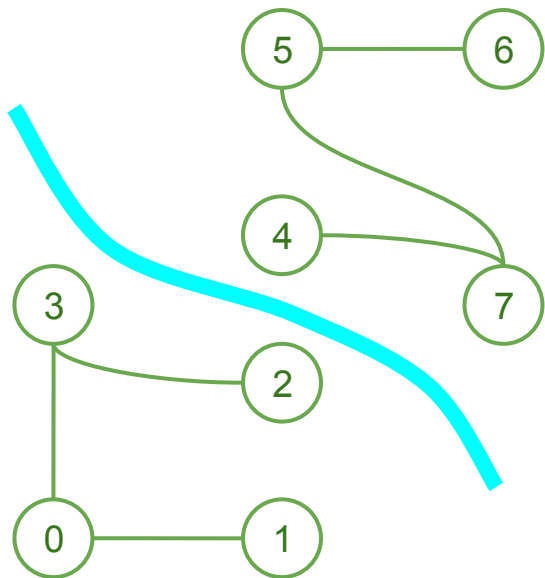
Working copy



Input graph



Working copy



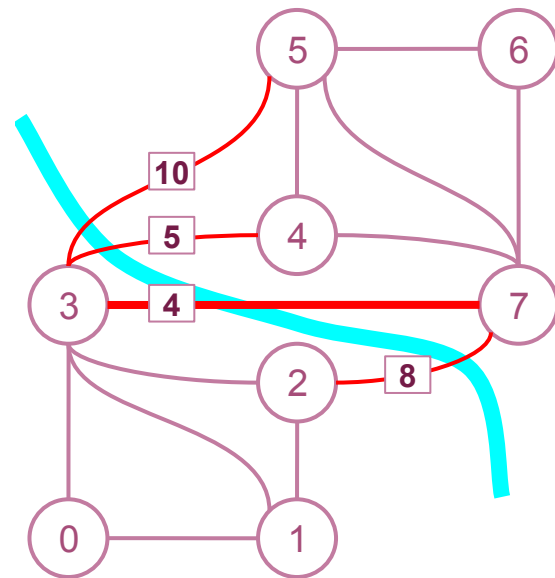
	0	1	2	3	4	5	6	7
2	∞	∞	∞	...	∞	∞	∞	∞
3	...	∞	...	∞	∞	∞	∞	∞

How to tell which vertices in the working copy are in different components?

Which of these vertices are adjacent in the input graph?

Which pair has the **smallest edge** in the input graph?

Input graph



	0	1	2	3	4	5	6	7
2	∞	...	∞	...	∞	∞	∞	8
3	...	...	...	∞	5	10	∞	4

Label vertices with
their components

How to tell which
vertices in the
working copy are in
different
components?

Which of these
vertices are adjacent
in the input graph?

Adjacency matrix
entry less than ∞

Greedy search

Which pair
has **the**
smallest
edge in the
input graph?

Label vertices with
their components

$$u_{\text{component}} \neq v_{\text{component}}$$

$$G[u][v] < \infty$$

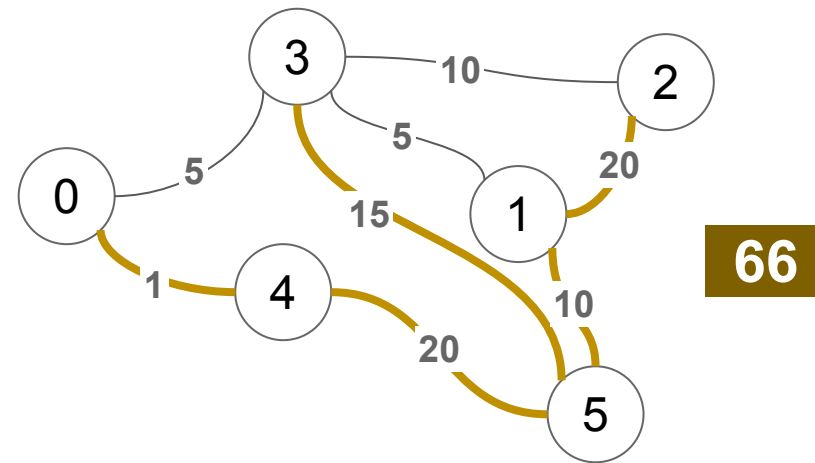
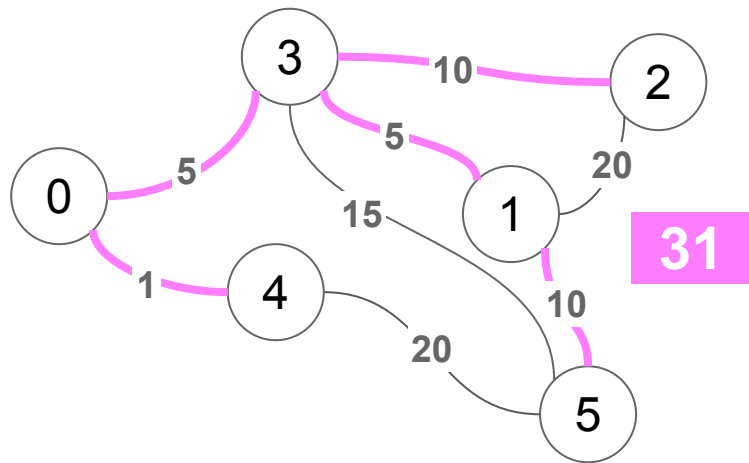
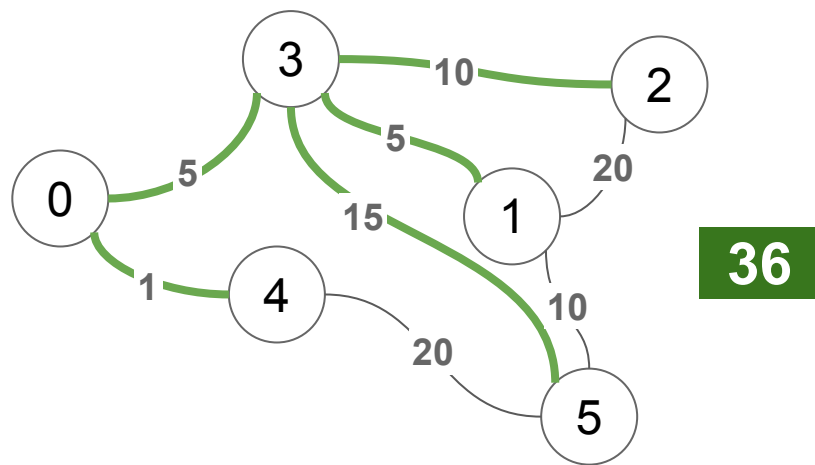
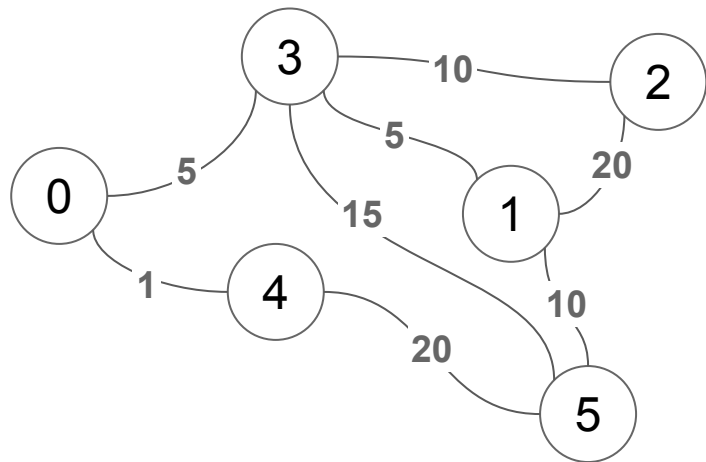
Adjacency matrix
entry less than ∞

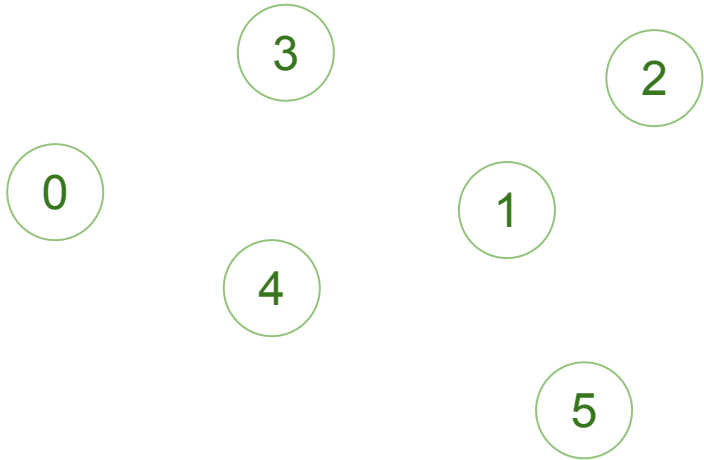
Greedy search smallest = ∞

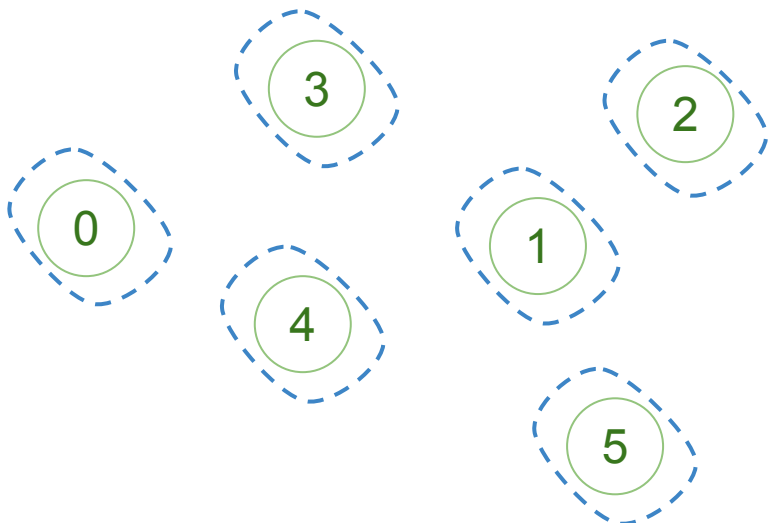
for every u, v in G

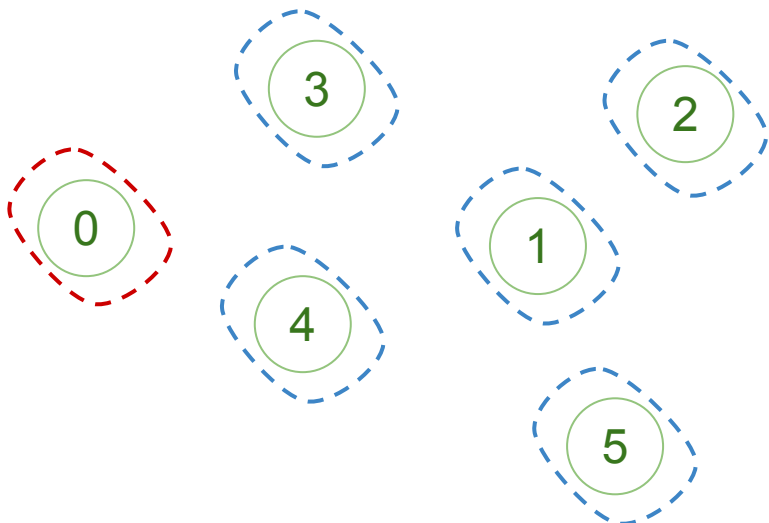
if $u_{\text{component}} \neq v_{\text{component}}$ and $G[u][v] < \infty$ and $G[u][v] < \text{smallest}$

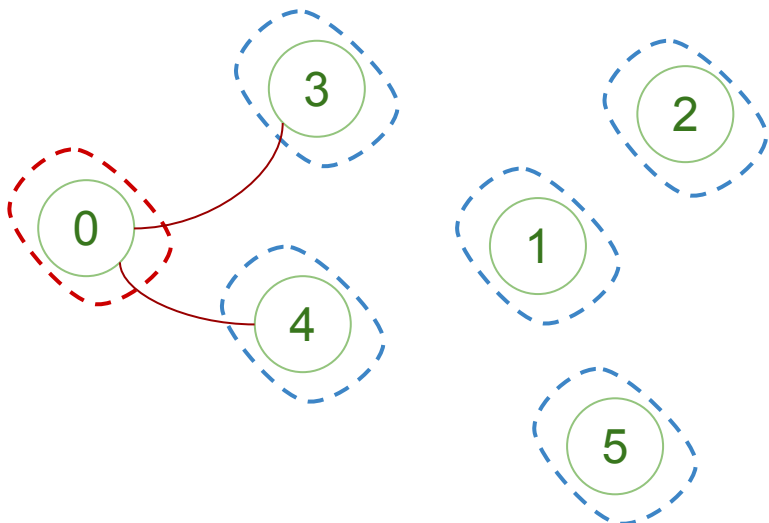
$$\text{smallest} = G[u][v]$$

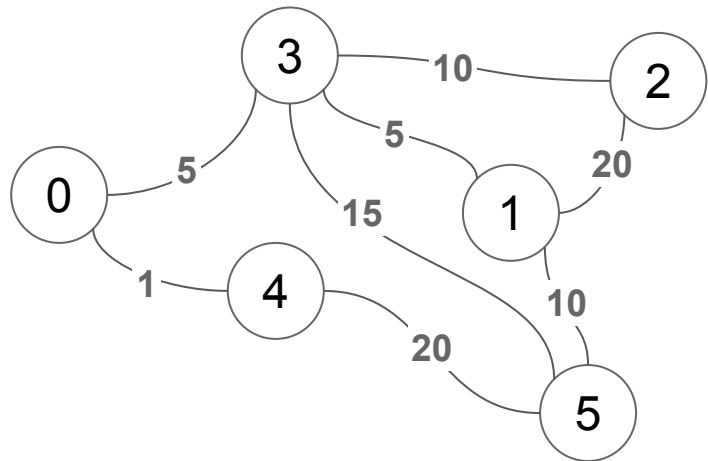


[illegible]

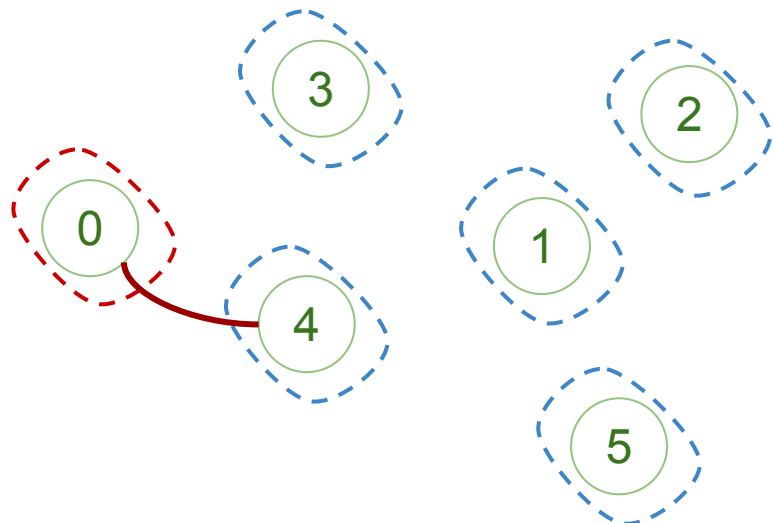
[illegible]

[illegible]

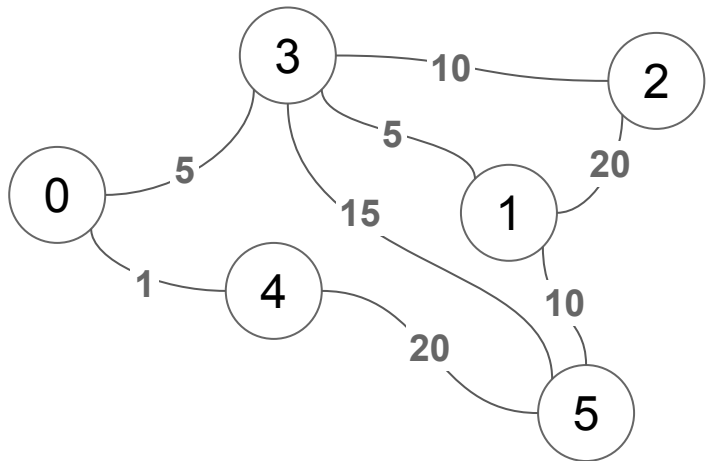
[illegible]



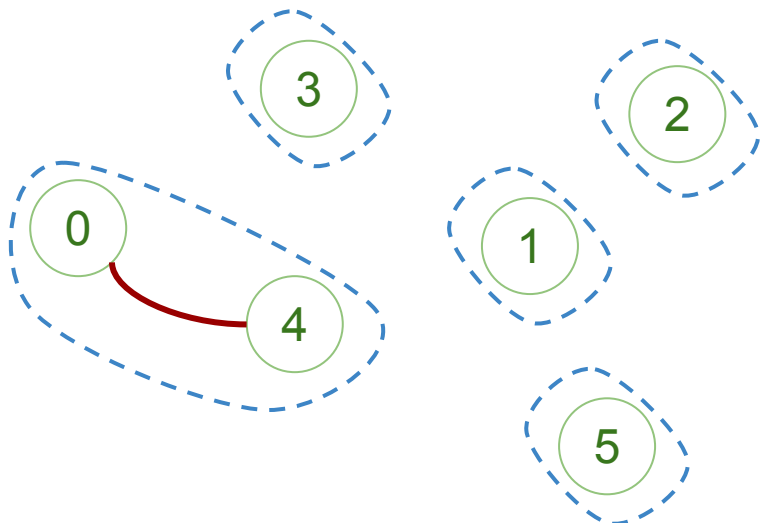
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



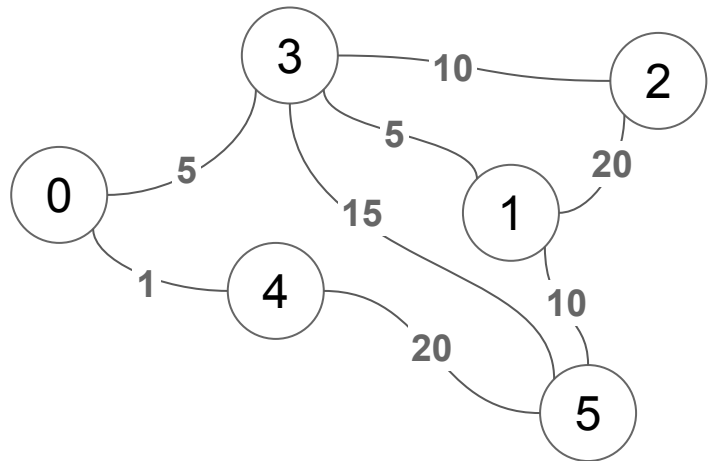
∞	∞	∞	∞	1	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



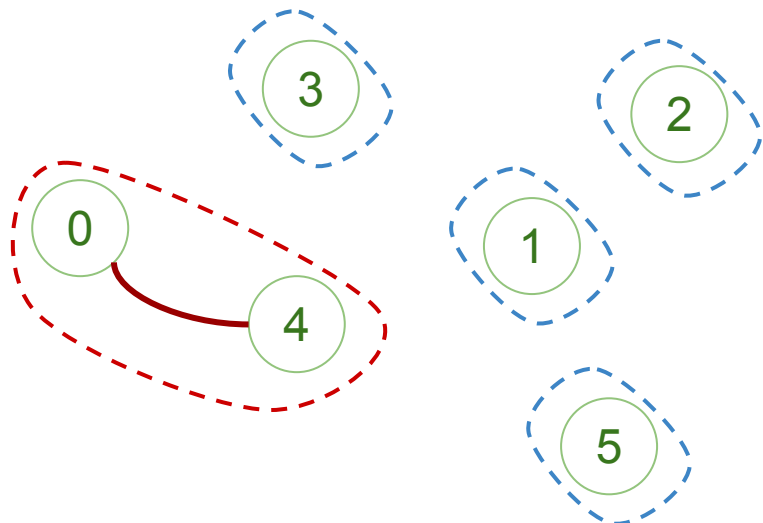
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



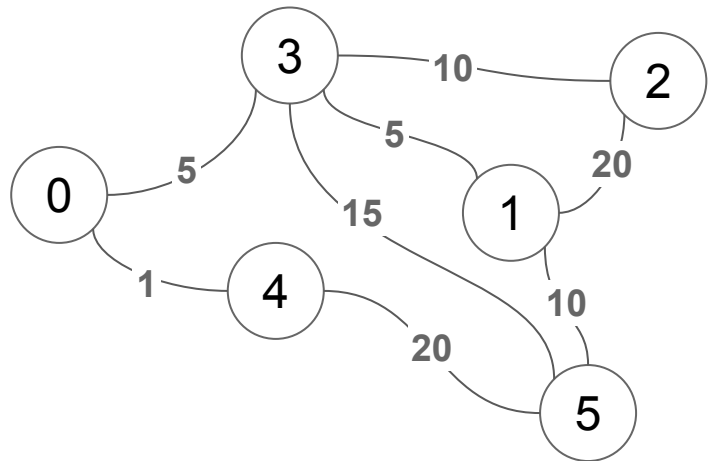
∞	∞	∞	∞	1	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



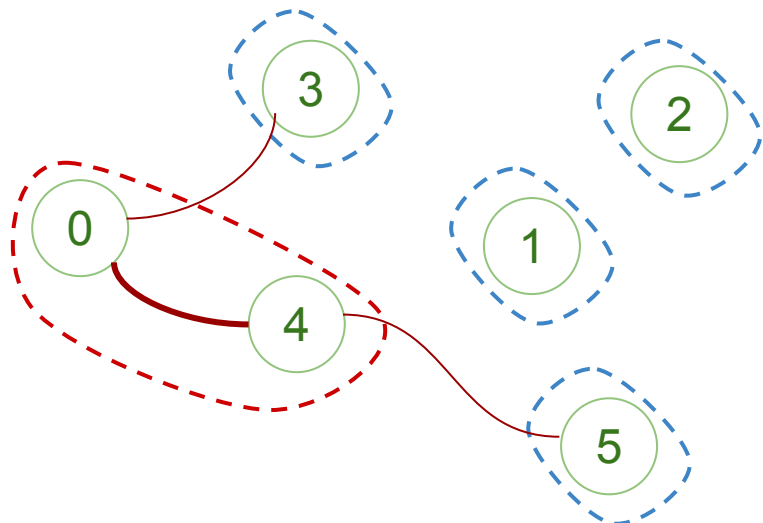
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



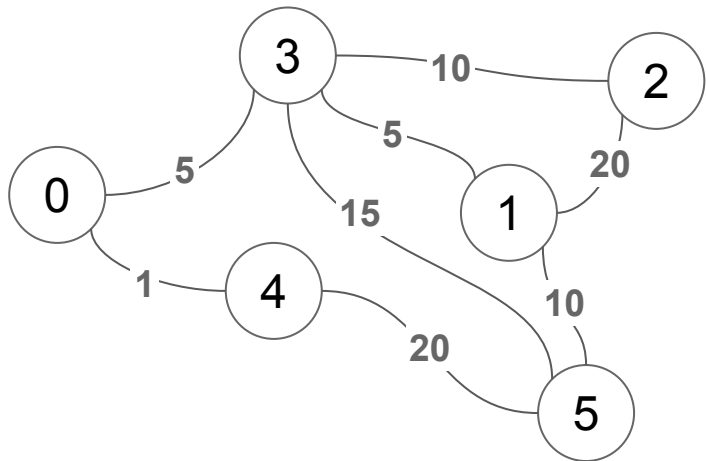
∞	∞	∞	∞	1	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



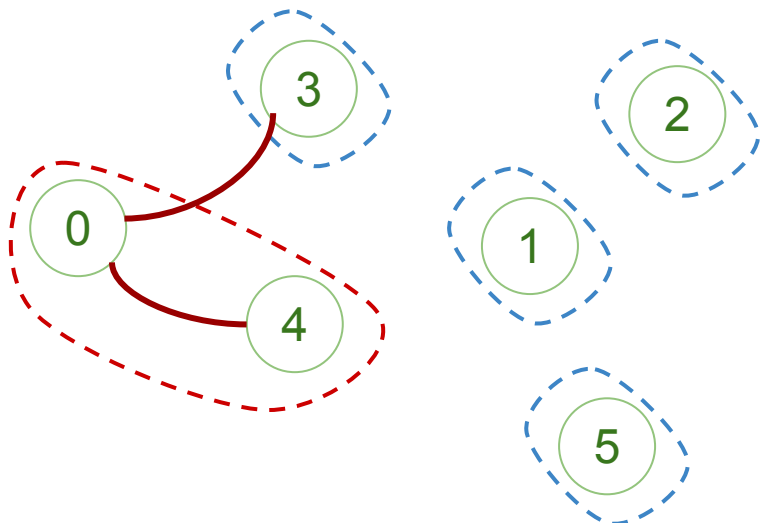
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



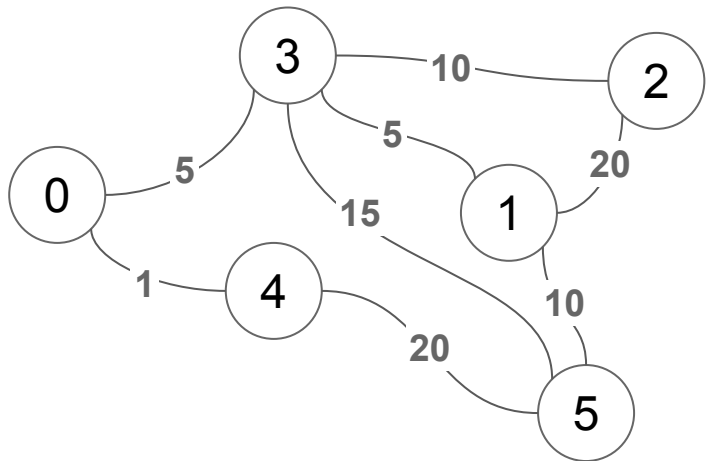
∞	∞	∞	∞	1	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



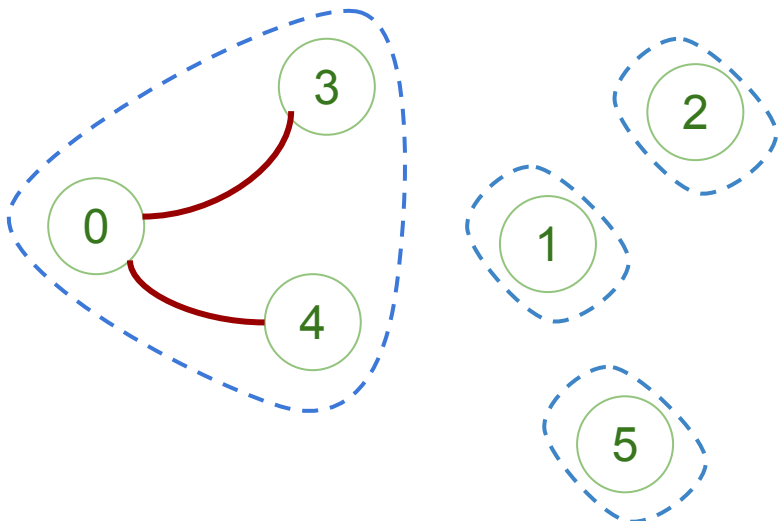
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



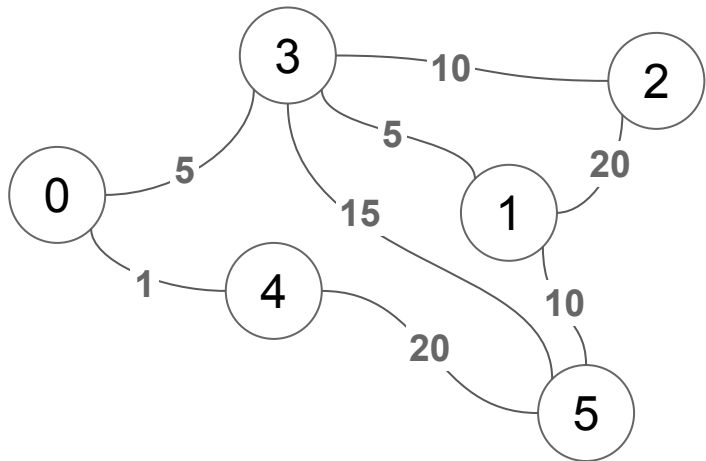
∞	∞	∞	5	1	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



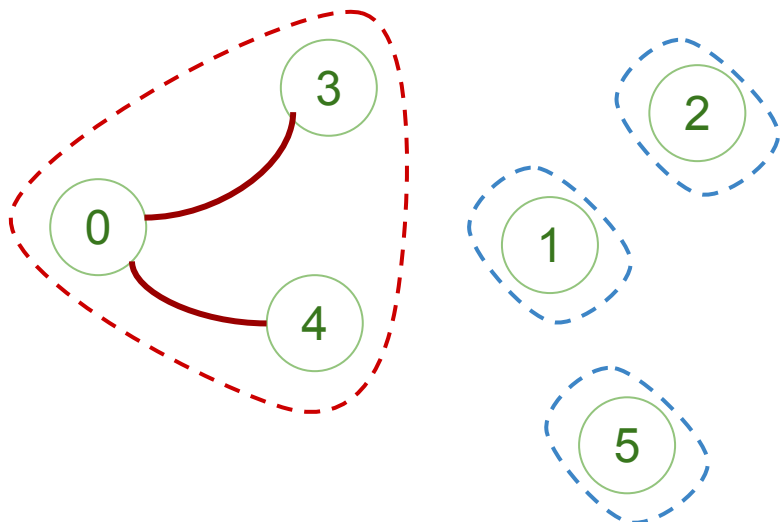
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



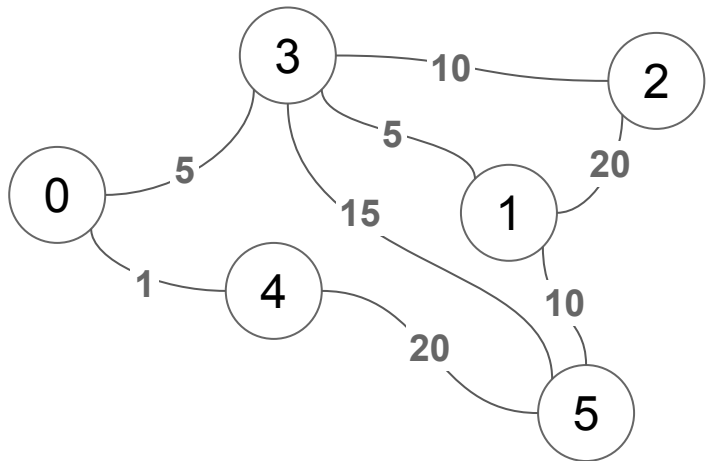
∞	∞	∞	5	1	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



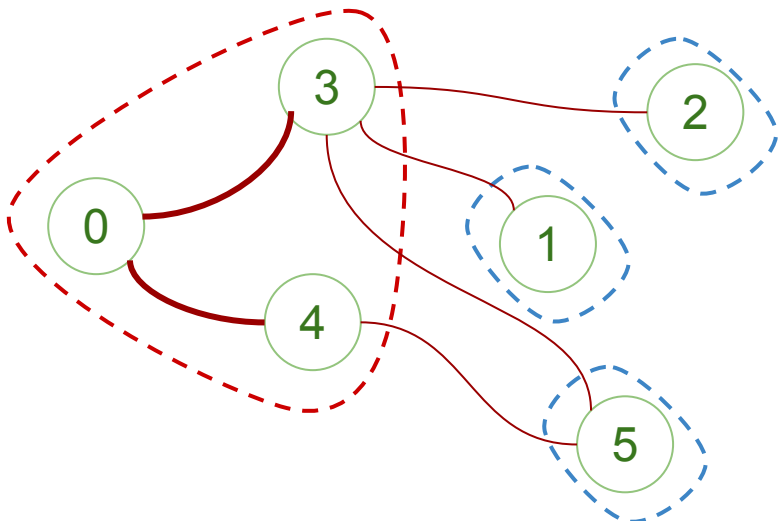
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



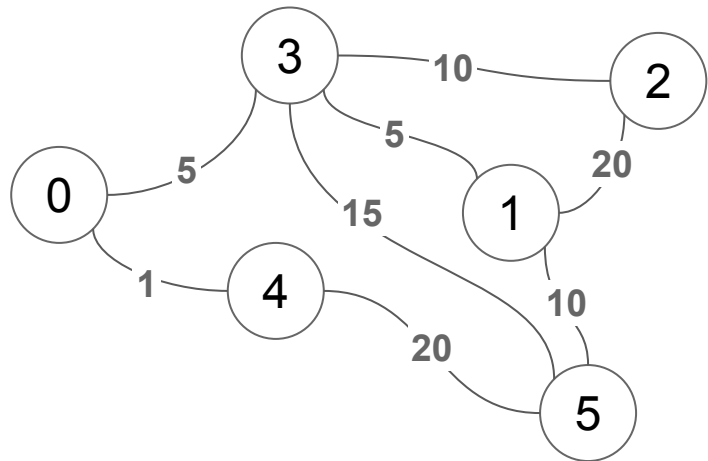
∞	∞	∞	5	1	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



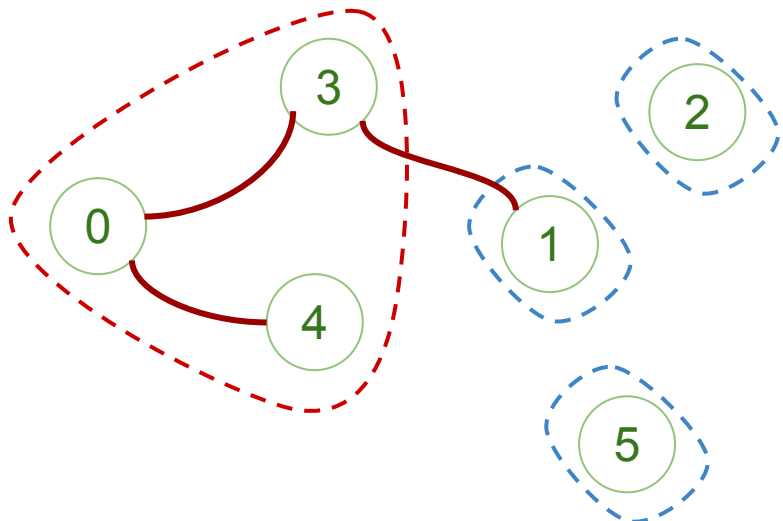
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



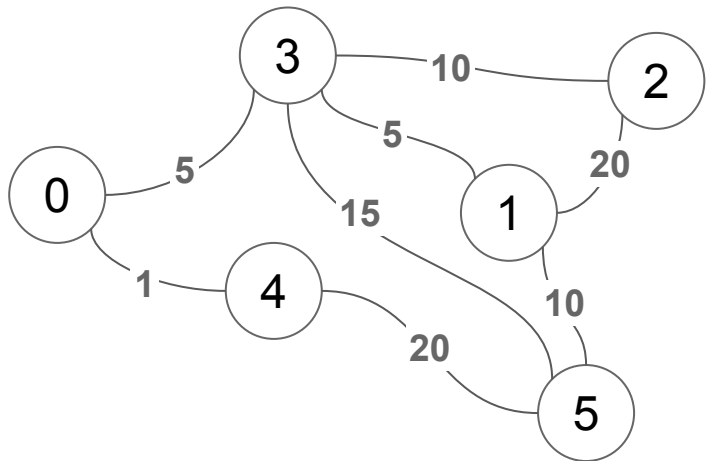
∞	∞	∞	5	1	∞
∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



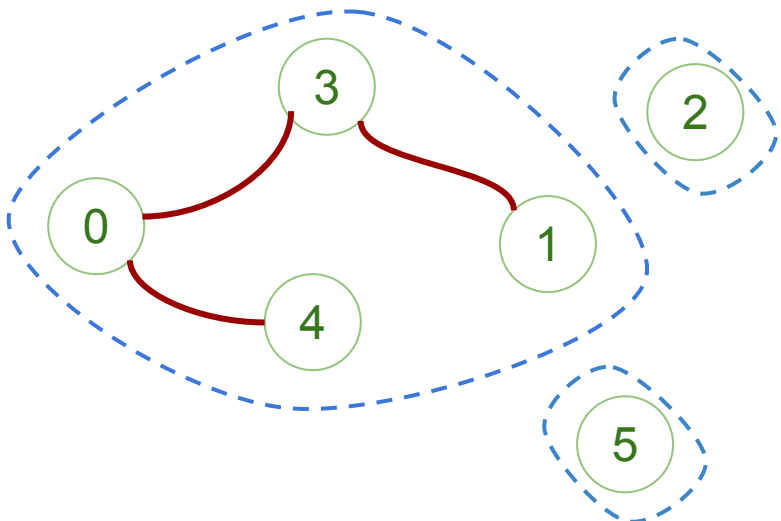
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



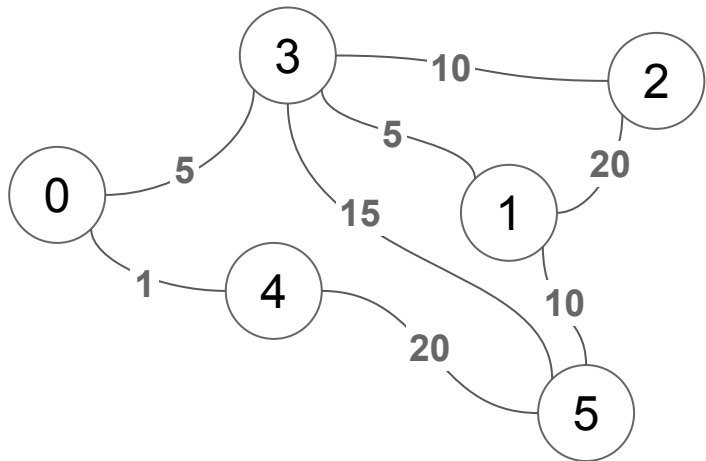
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	∞	∞	∞
5	5	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



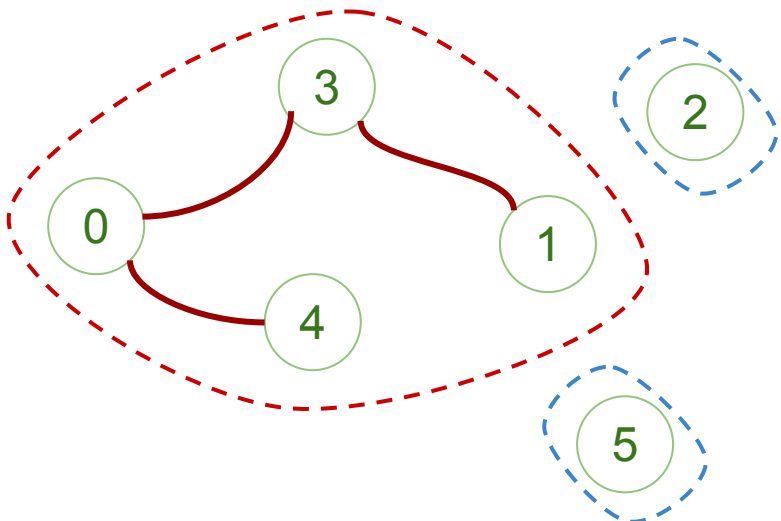
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



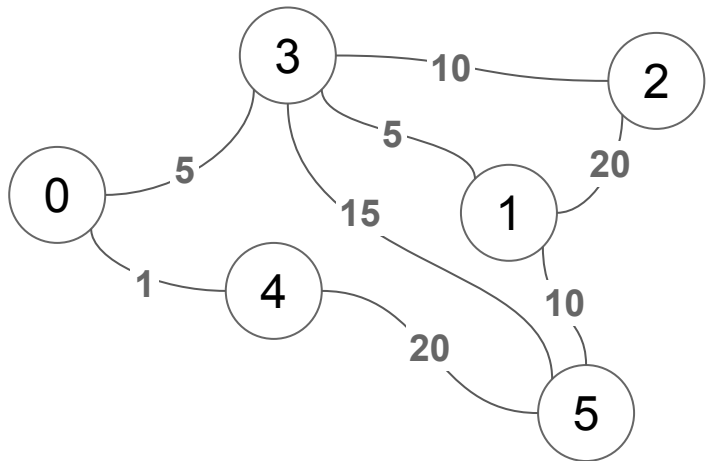
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	∞	∞	∞
5	5	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



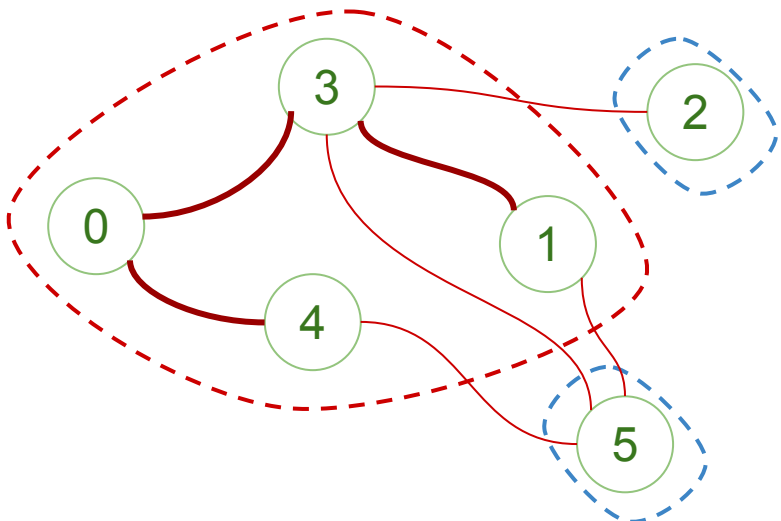
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



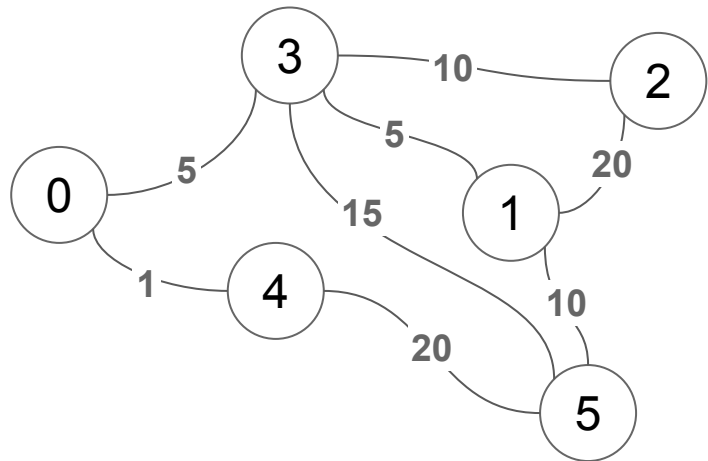
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	∞	∞	∞
5	5	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



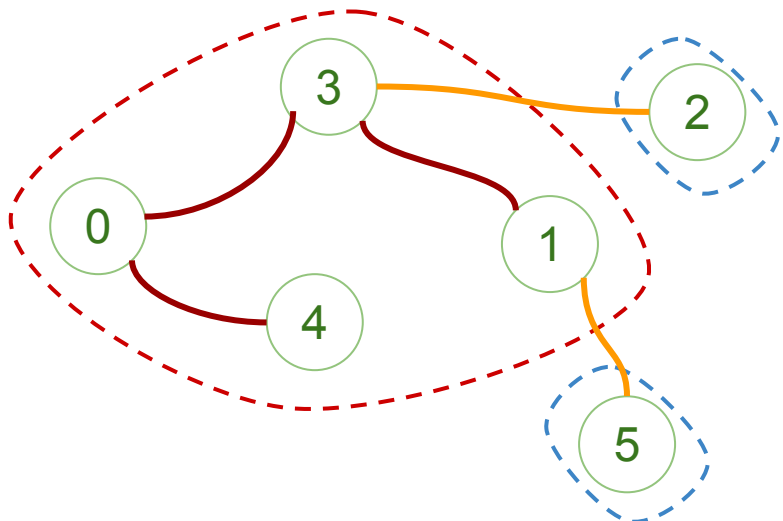
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



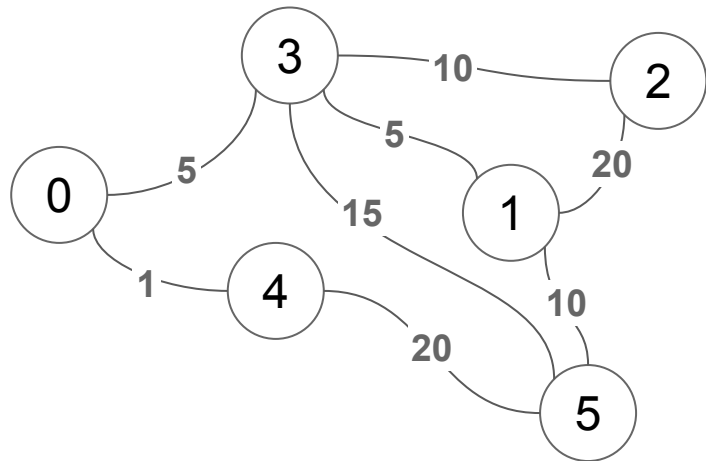
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	∞	∞	∞
5	5	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



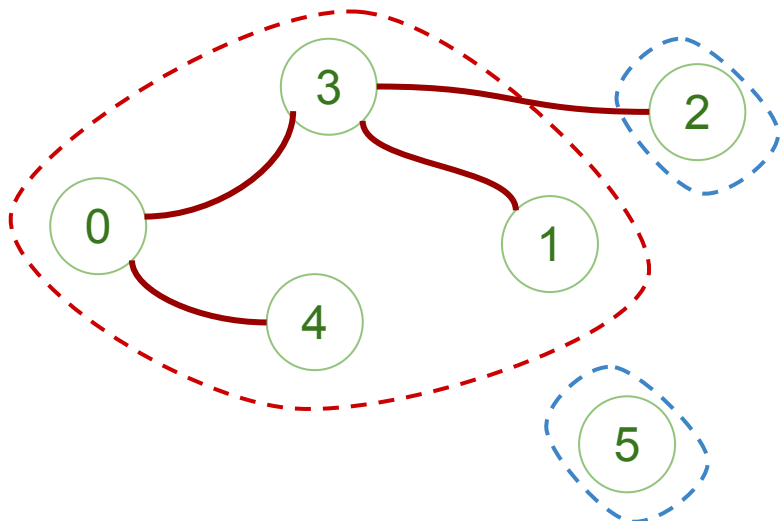
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



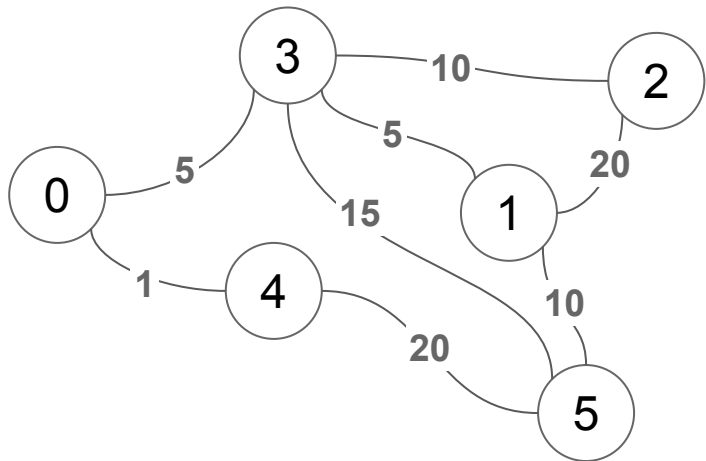
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	∞	∞	∞
5	5	∞	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



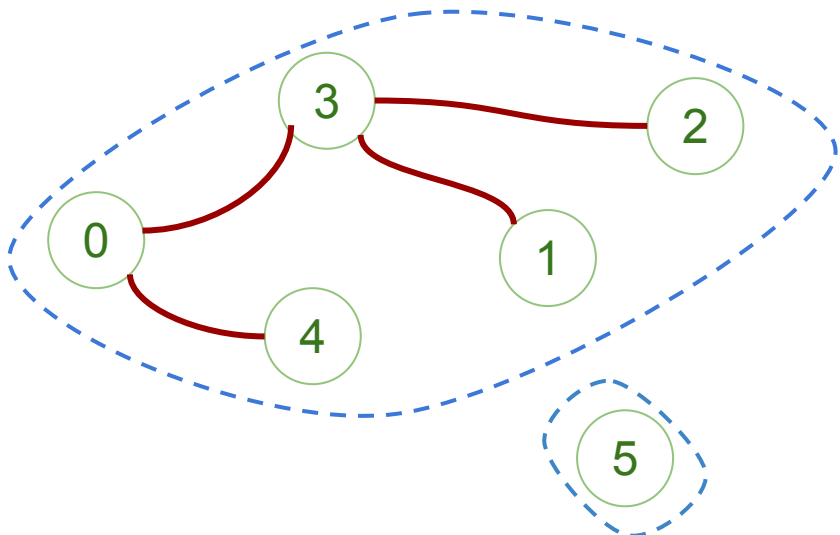
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



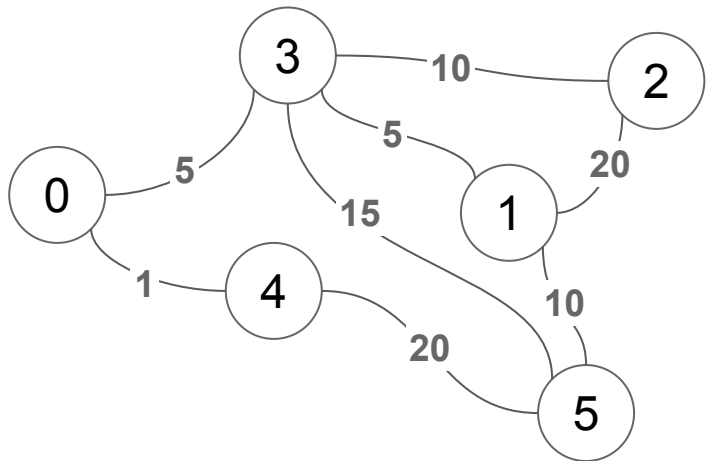
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	10	∞	∞
5	5	10	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



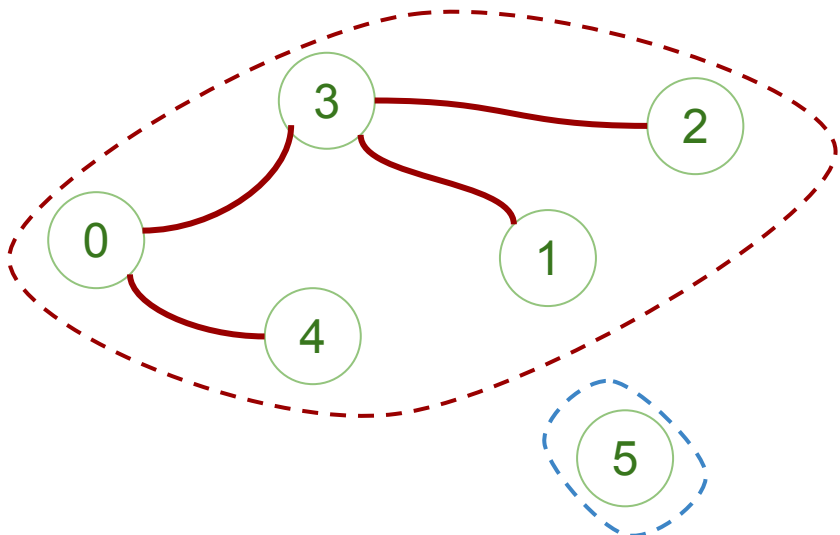
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



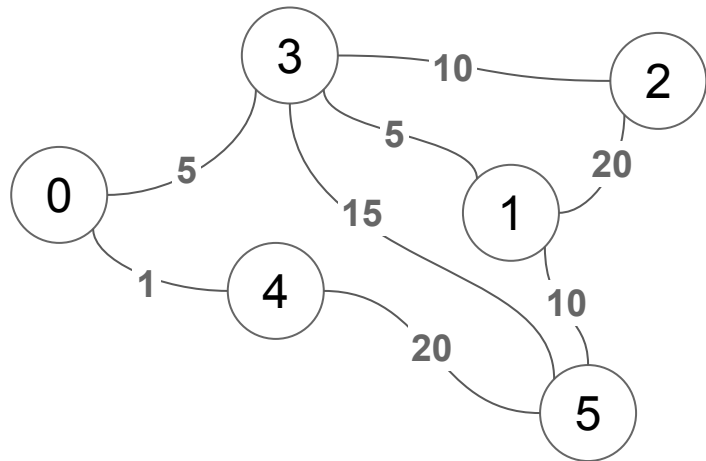
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	10	∞	∞
5	5	10	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



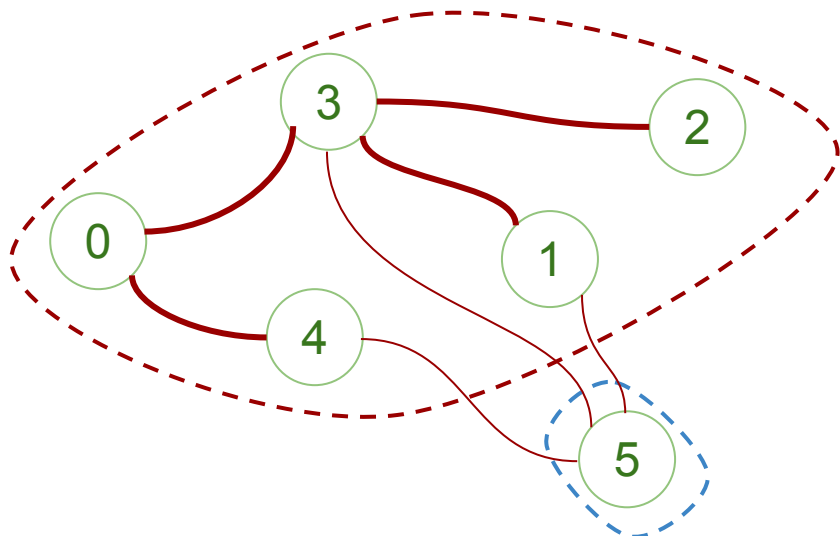
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



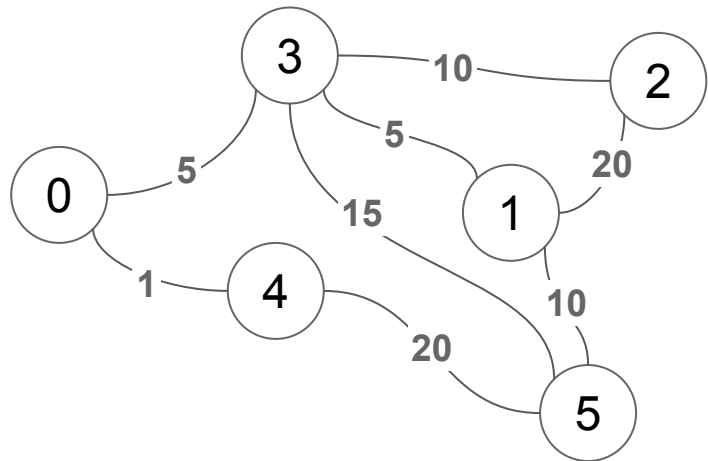
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	10	∞	∞
5	5	10	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



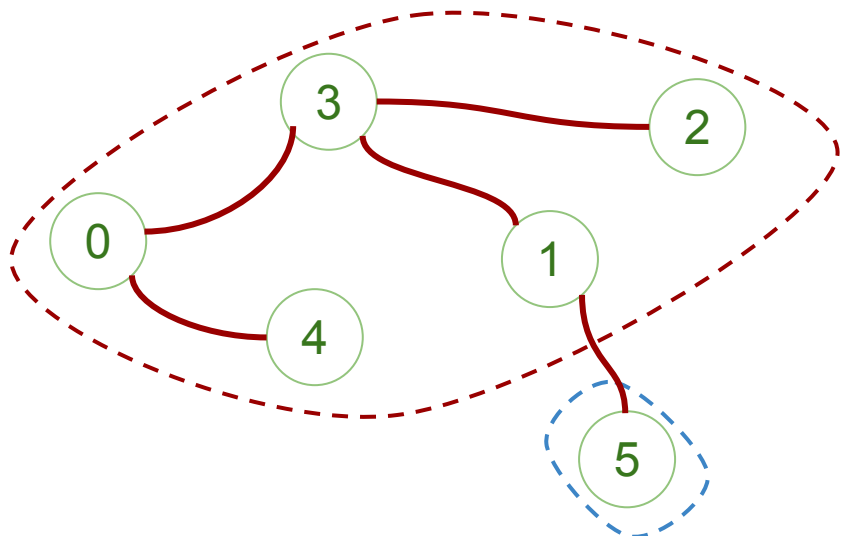
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



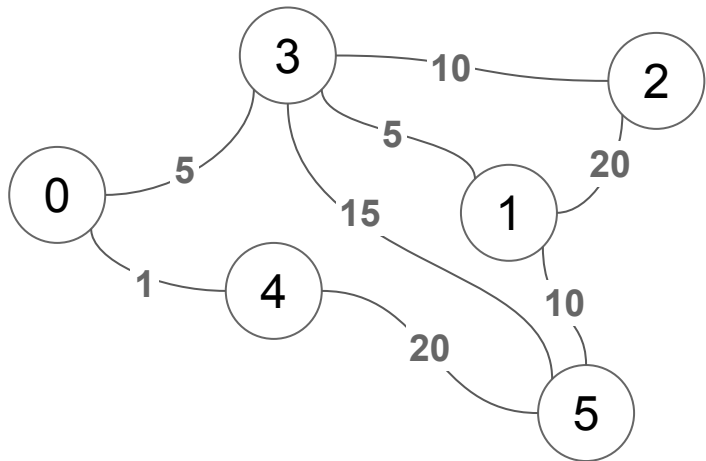
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	10	∞	∞
5	5	10	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



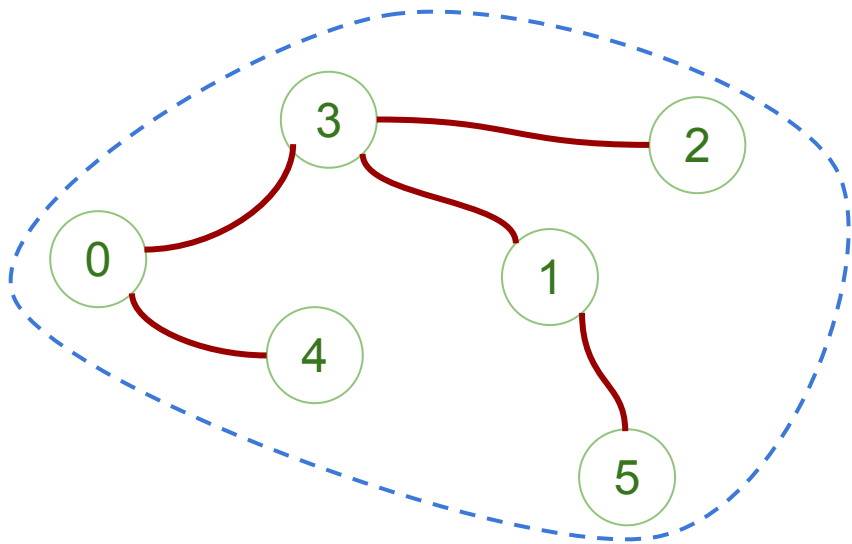
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



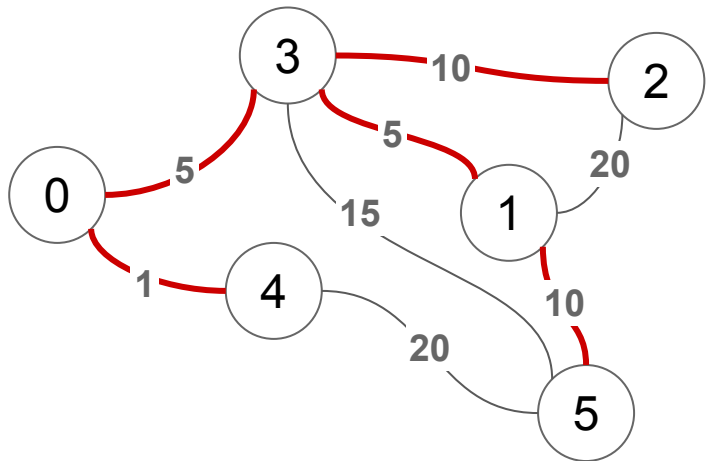
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	10
∞	∞	∞	10	∞	∞
5	5	10	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	10	∞	∞	∞	∞



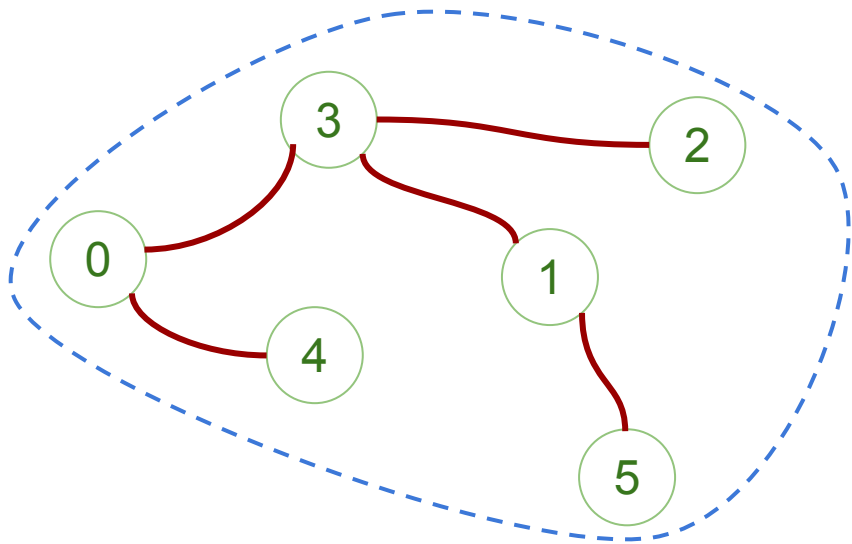
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	10
∞	∞	∞	10	∞	∞
5	5	10	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	10	∞	∞	∞	∞

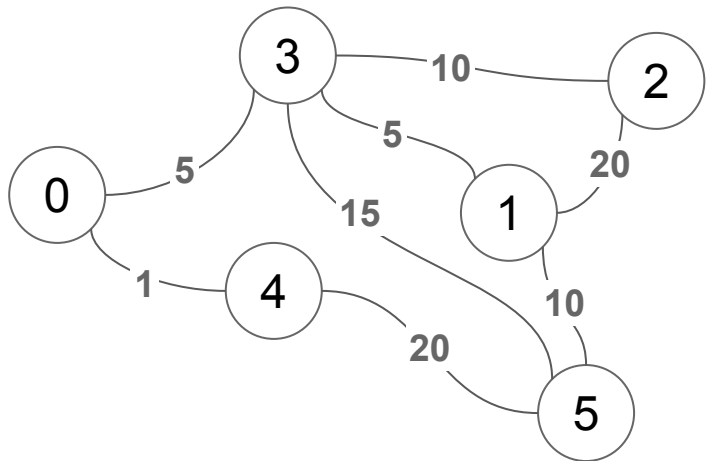


∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	10
∞	∞	∞	10	∞	∞
5	5	10	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	10	∞	∞	∞	∞

Data Structures and Code


$$G =$$

∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞


$$T =$$
[illegible]

Copying input G ...

```
import copy
T = copy.deepcopy(G)
```

... to candidate
minimum spanning tree T

$$G =$$

∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞

Number of vertices:
 $\text{len}(G) = \text{len}(T)$

$$T \equiv$$
[illegible]

mark all vertices in T as not visited

for every vertex v in T

if v not visited:

```
component count += 1
```

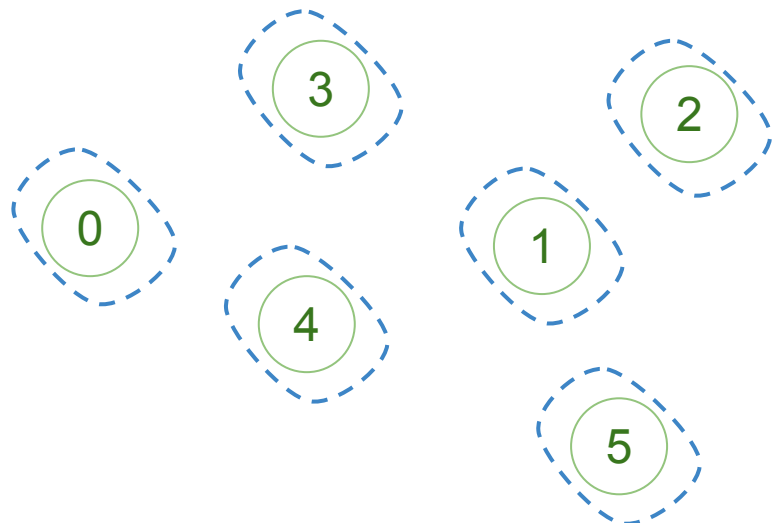
r = all vertices reachable from **v**

label $\mathbf{v}, \mathbf{r}$ with component count

add v, r to visited vertices

Counting components

and labeling

[illegible]

mark all vertices in T as not visited
for every vertex v in T

if v not visited:

component count += 1

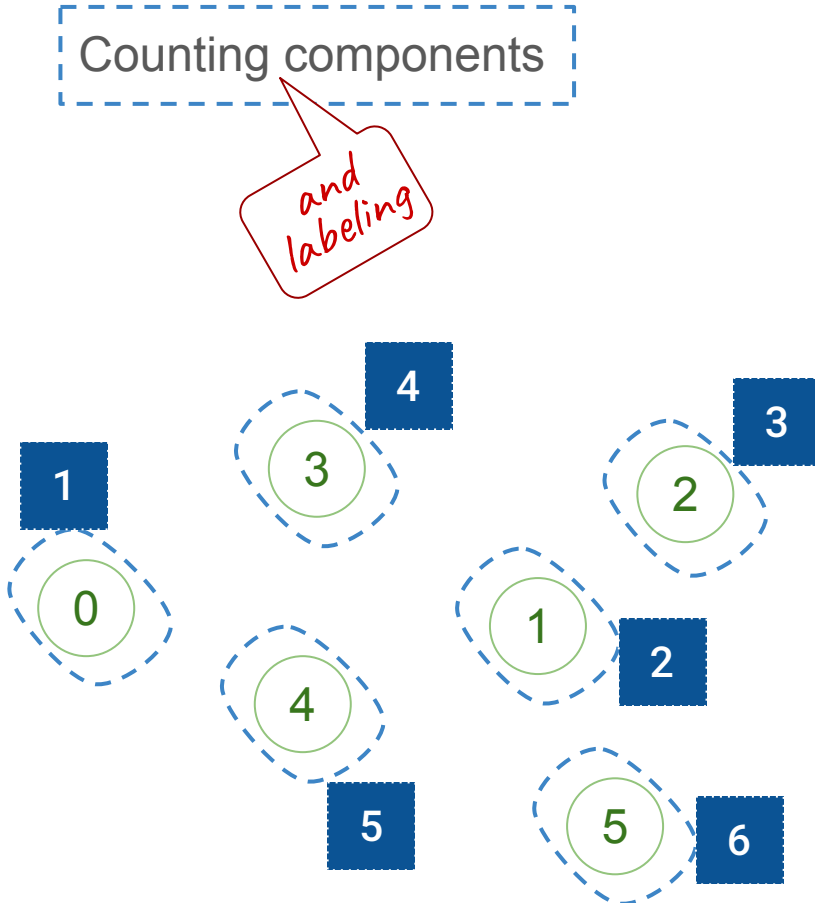
r = all vertices reachable from v

label v, r with component count

add v, r to visited vertices

Look up table
for vertex:component

1
2
3
4
5
6



mark all vertices in T as not visited
for every vertex v in T

if v not visited:

component count += 1

r = all vertices reachable from v

label v, r with component count

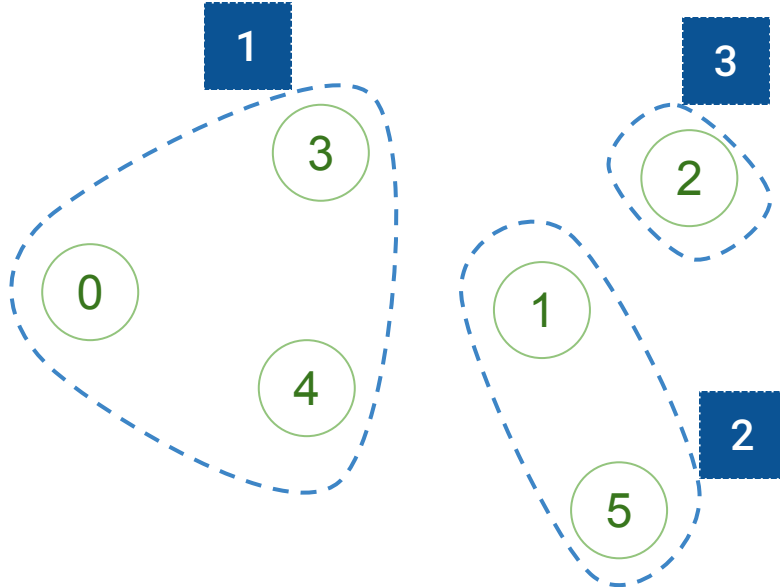
add v, r to visited vertices

Look up table
for vertex:component

1
2
3
1
1
2

Counting components

and
labeling



mark all vertices in T as not visited
for every vertex v in T

if v not visited:

component count += 1

r = all vertices reachable from v

label v, r with component count

add v, r to visited vertices

Look up table
for vertex:component

1
1
1
1
1
1

Counting components

and
labeling

1

3

2

0

1

4

5

A *simple* algorithm on candidate min. spanning tree T

while T has > 1 components:

 choose any two components

 connect them via a **safe edge**

 count and label components

A *simple* algorithm on candidate min. spanning tree T

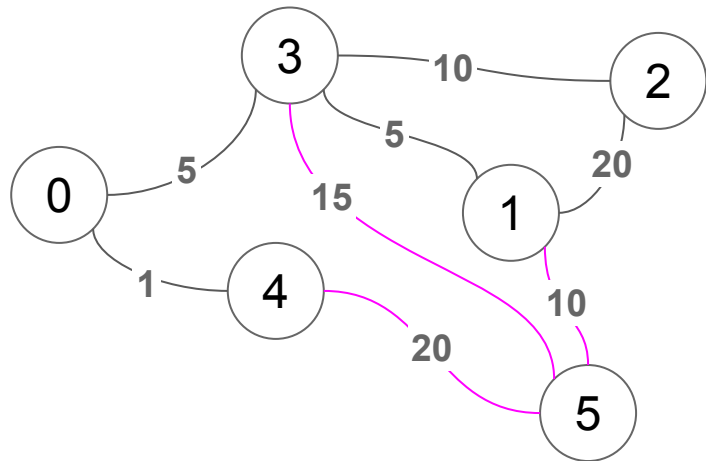
add **safe edges** and repeat

safe edge

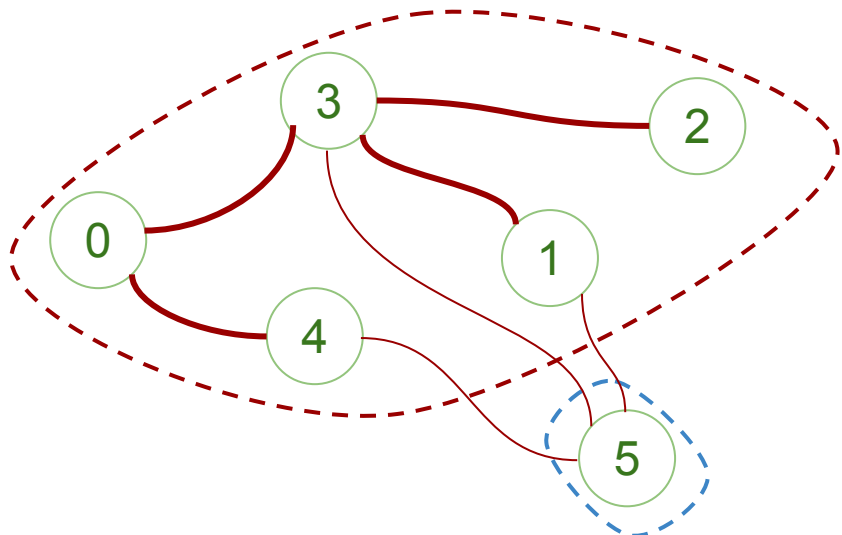
A *hypothetical* edge between two components.

When added to T , it merges the two components.

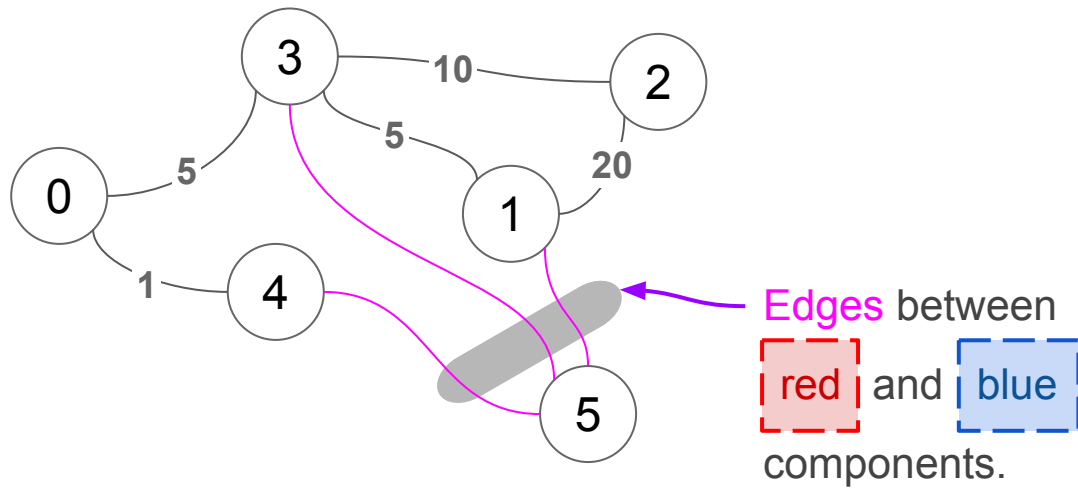
Selected among **specific** edges in G .



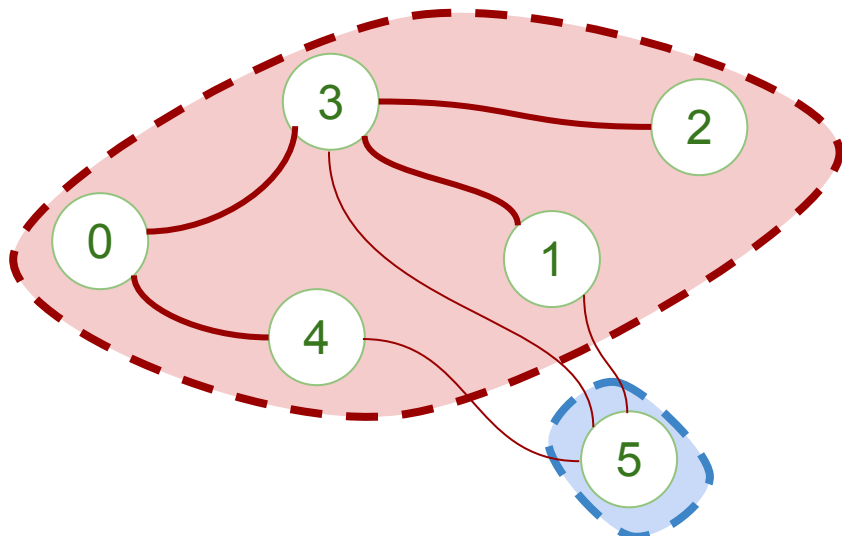
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



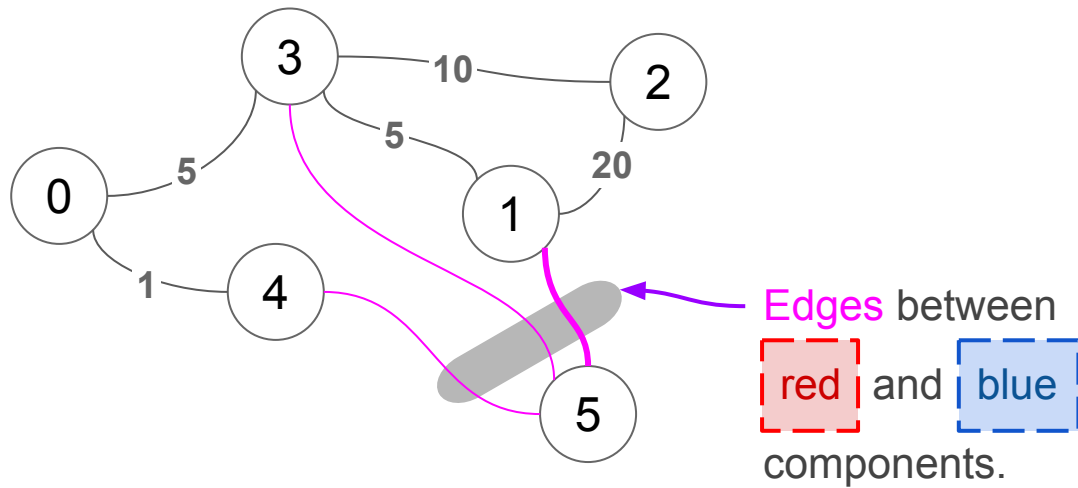
∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	10	∞	∞
5	5	10	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



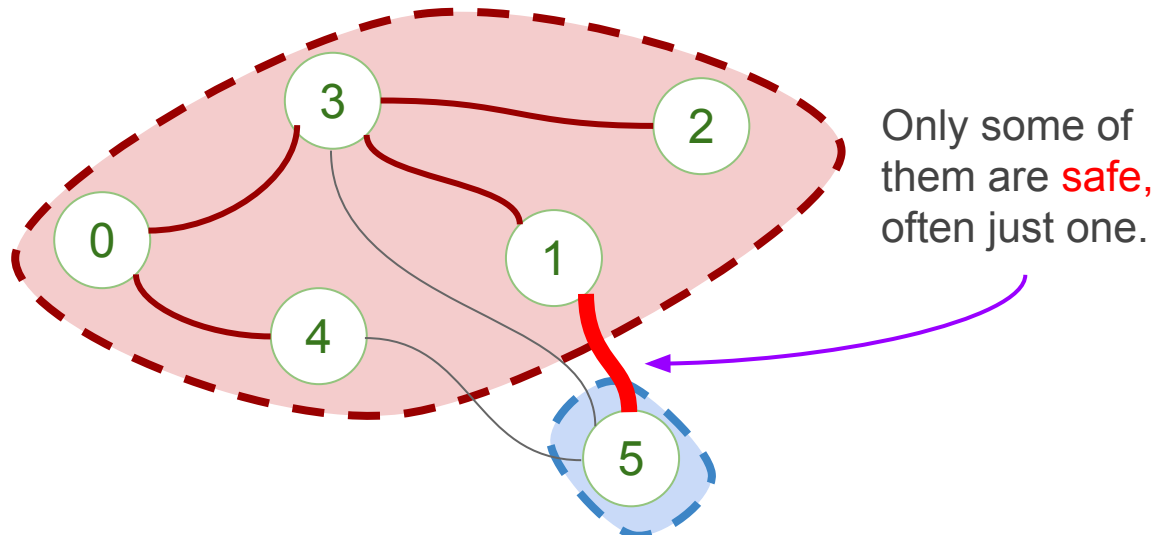
∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	∞
∞	∞	∞	10	∞	∞
5	5	10	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞



∞	∞	∞	5	1	∞
∞	∞	20	5	∞	10
∞	20	∞	10	∞	∞
5	5	10	∞	∞	15
1	∞	∞	∞	∞	20
∞	10	∞	15	20	∞



∞	∞	∞	5	1	∞
∞	∞	∞	5	∞	10
∞	∞	∞	10	∞	∞
5	5	10	∞	∞	∞
1	∞	∞	∞	∞	∞
∞	10	∞	∞	∞	∞

Work flow

Input: adjacency matrix G

T = edgeless copy of G

while T has more than 1 components:

- select a component from T

- find a safe edge out of that component

- connect the two components via safe edge

- recount components

return all safe edges

Work flow

Input: adjacency matrix G  All the work is done in here.

T ← edgeless copy of G

while T has more than 1 components:

 select a component from T

 find a safe edge out of that component

It leads to
another
component in T 

 connect the two components via safe edge

 recount components

There should be
at least one less
component 

return all safe edges

 These are the edges
of the MST

Work flow

Input: adjacency matrix G

T = edgeless copy of G

while T has more than 1 components:

 select a component from T

 find a safe edge out of that component

 connect the components via safe edge

 recount components

return all safe edges

Need to count (and label) components

count_label(graph):

 no_edge = graph[0][0]

 n = len(graph)

 count = 0

 comp = [-1] * n

 marked = []

 for s in range(len(n)):

 if s not in marked:

 count += 1

 reached = reach(s, graph)

 for v in reached:

 comp[v] = count

 marked += reached

 return count, comp

Work flow

Input: adjacency matrix G

T = edgeless copy of G

while T has more than 1 components:

 select a component from T

 find a safe edge out of that component

 connect the components via safe edge

 recount components

return all safe edges

How?

naive

```
for u in range(n):  
    for v in range(n):  
        if comp[u] != comp[v]:  
            # u, v in different components
```

How?

no easy way out ...

```
safe[ comp[ u ] ] = # either None or a list  
                   # like [i,j] where i, j are  
                   # vertex labels such that  
                   # comp[i] = comp[u] and  
                   # comp[j] != comp[u]
```